

Verification Continuum™

Verdi® SystemC Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	6
Synopsys Statement on Inclusivity and Diversity	7

1. Setup and Requirements	8
Debugger Setup With VCS	8
Unified Front-end Post-Process Debug	9
Unified Front-end Post-Process Debug With -kdb=sysc_elab Option	10
Unified Front-end Post-Process Debug With -cbug::invokeElabcom Option ...	10
UFE Interactive Debug	11
UFE Interactive Debug With -kdb=sysc_elab Option	11
Unified Front-end Post-Process Debug With -kdb=sysc_blackbox_elab Option	11
UFE Interactive Debug With -kdb=sysc_blackbox_elab Option	12
Debugger Setup With IUS	12
Set up a Directory for SystemC Database	13
Supported TCL Command	14
deblImport	14
Simulator Setup With Questasim	14
Generate FSDB	15
Dump FIFO to FSDB	16
Dump Analog Signals in MSV	16
Setting the FIFO Size Environment Variable	17
Create the RIDB File	17

2. Support Scopes	19
Supported Platforms and Simulator	23

3. Usage	24
Main Window	24
Hierarchy Browser	24
Signal List	25
OSCI-name versus Source-name	27

Contents

Source Code	28
nSchema	30
Showing FSDB	30
Search Signals	31
Special Appearance in nWave	32
Highlight Signals	33
Temporal Flow View	34
nMemory	34
4. Interactive Simulation Debug	36
Starting SystemC Interactive Debugging	36
Invoking Verdi Interactive Debug Mode With Unified Compiler Front-End	37
Opening the C Source Code File	38
Limitation	40
Enabling C/C++ Debugging in Interactive Debug Mode	40
Simulation Control Commands	42
C-Source Debugger (CBug) Functions	43
next -hdl	43
Cross-step into SC Processes	43
Step Through SystemC	43
Cross-Step between SV and PLI/DPI	44
Direct GDB Commands	44
Local Tab	45
Change Value	46
Setting the Radix of C Variables	49
Filter and Search Functions	50
Watch Tab	51
Filter and Search Functions	52
Stack Pane	52
Dynamic Dumping	53
Dump and Get Signals	54
Dumping Options	54
Variable Value Tip	55
Setting Value-Change Breakpoint on C Variables	56
Verdi GUI Usage	56

Contents

Setting a Value-Change Break Point on a C-Variable	58
UCLI Mode Usage	59
Execution stops on reaching the next line	60
Expressions and Runtime Overhead	60
Scope of the Expressions	60
C-VCBP States	60
Reactivating a C-VCBP with a Different Breakpoint	61
Associated Breakpoints	62
Creating a C-VCBP and Line Breakpoint Pair	63
-location option	64
-C_expression option	65
VCBP of Memory Locations	65
False Positives of C-VCBPs	66
C-VCBP Restart Behavior	66
Supported Data Types and Hardware Limitations	67
5. Known Problems and Limitations	69

Preface

The Verdi® Automated Debug Platform provides debug support for mixed HDL and SystemC designs. You can dump SystemC signals, view SystemC instances hierarchy, step through the SystemC/C++ source code, and debug the interactions between the SystemC code and the HDL code.

In addition, you can also debug DPI, PLI, or any other C/C++ code linked to the simulation.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to support_center@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to support_center@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Setup and Requirements

The following options must be set to use Verdi SystemC Debug features:

1. Setup simulators for enabling the SystemC flow.

Refer to the [Debugger Setup With VCS](#), [Debugger Setup With IUS](#), and [Simulator Setup With Questasim](#) sections for details.

2. Generate the FSDB file (optional).

Refer to the [Generate FSDB](#) section for details.

3. Prepare the RIDB file before performing debugging.

Refer to the [Create the RIDB File](#) section for details.

The following sections describe how to:

1. Set up the environment variable.
2. Set up the requirements of the corresponding simulator.
3. Generate the FSDB and RIDB files.

Instructions to invoke Verdi with SystemC design and to invoke it with Unified Compiler Front-end are also described.

Debugger Setup With VCS

Perform the following sections to compile the SystemC source files and any other C/C++ file that must be debugged:

- [Unified Front-end Post-Process Debug](#)
- [Unified Front-end Post-Process Debug With -kdb=sysc_elab Option](#)
- [Unified Front-end Post-Process Debug With -cbug::invokeElabcom Option](#)
- [UFE Interactive Debug](#)
- [UFE Interactive Debug With -kdb=sysc_elab Option](#)

- [Unified Front-end Post-Process Debug With -kdb=sysc_blackbox_elab Option](#)
- [UFE Interactive Debug With -kdb=sysc_blackbox_elab Option](#)

Unified Front-end Post-Process Debug

Analyze the Verilog/VHDL and System C/C++ files with the `-kdb` flag. Add this flag also during VCS elaboration. This invokes Unified Compiler Front-end (UFE).

1. Add the `-cflags -g` option during the analysis for getting the debug information for SystemC, C, and C++ source files.

- `syscan ... -cflags -g ...`

2. Elaborate `-kdb` and `-debug_access+pp` options.

For example,

```
% syscan -kdb -cflags -g <the SystemC files>
% vlogan -kdb <the Verilog files>
% vcs -sysc <top_module_name> -kdb -debug_access+pp
% ./simv -ucli -do dump.tcl
% verdi -ssf novas.fsdb -dbdir simv.daidir
```

The `dump.tcl` file contains the following commands:

```
fsdbDumpvars
run
```

Note:

Generating RIDB and elaborating the KDB database is delayed in this process when Verdi is invoked for the first time causing a noticeable delay up to several minutes. This delay happens only when the design contains SystemC. There are several possible approaches to avoid this delay when you invoke Verdi for the first time:

- Use the VCS `-kdb=sysc_elab elab` option (interactive and post-process)
- Use `cbug::invokeElabcom` command (post-process only)
- Blackbox SystemC

In the UFE flow, when SystemC related design is imported, if the runtime database (`simv.daidir/default.ridb`) is not generated, then Verdi invokes simulator to dump RIDB. In post-processing mode, the console window appears which is the same as in interactive mode. You can see the fail reason in the Console directly even in post-processing mode.

Unified Front-end Post-Process Debug With -kdb=sysc_elab Option

Add the `-kdb=sysc_elab` option in the compilation command line. VCS triggers creating RIDB and KDB elaboration when it is compiling `simv`. This makes the VCS compilation slower but there is no delay when you later invoke Verdi.

The runtime arguments of `simv`, the environment variable, and the input files must be provided accurately. If the attempt to create RIDB and KDB elaboration fails, then a warning message is issued and you experience a delay when you invoke Verdi for the first time.

For example,

```
% syscan -kdb -cflags -g <the SystemC files>
% vlogan -kdb <the Verilog files>
% vcs -sysc <top_module_name> -kdb=sysc_elab -debug_access+pp
% ./simv -ucli -do dump.tcl
% verdi -ssf novas.fsdb -dbdir simv.daidir
```

The `dump.tcl` file contains the following commands:

```
fsdbDumpvars
run
```

Unified Front-end Post-Process Debug With -cbug::invokeElabcom Option

Call the UCLI command `-cbug::invokeElabcom` option at the same time when you call `fsdbDumpvars` command. Creating RIDB and KDB elaboration happens when `simv` runs, so there is no delay when you later invoke Verdi.

For example,

```
% syscan -kdb -cflags -g <SystemC Files>
% vlogan -kdb -sverilog <verilog_files>
% vcs -sysc sc_main -kdb -debug_access+pp
% ./simv.vdb -ucli -do invokeElab.tcl
% verdi -ssf novas.fsdb -dbdir simv.daidir
```

The `invokeElab.tcl` contains the following commands:

```
fsdbDumpvars
cbug::invokeElabcom run
```

Note:

RIDB generation is performed at runtime and hence Verdi is invoked faster when invoked for the first time.

UFE Interactive Debug

When the KDB is generated using UFE, you can invoke Verdi with the KDB through `-verdi` simulator command-line options to perform interactive simulation debugging without other configurations:

For example,

```
% syscan -kdb <the SystemC files> -cflags -g
% vlogan -kdb <the Verilog files>
% vcs -sysc <top_module_name> -kdb -debug_access+all
% ./simv -verdi
```

UFE Interactive Debug With `-kdb=sysc_elab` Option

The runtime arguments of `simv`, the environment variable, and the input files must be provided accurately. If the attempt to create RIDB and KDB elaboration fails, then a warning message is issued and you experience a delay when you invoke Verdi for the first time.

For example,

```
% syscan -kdb <the SystemC files>
% vlogan -kdb <the Verilog files>
% vcs -sysc <top_module_name> -kdb=sysc_elab -debug_access+all
% ./simv -verdi
```

Unified Front-end Post-Process Debug With `-kdb=sysc_blackbox_elab` Option

If the design contains SystemC which is instantiated with Verilog/VHDL and debugging the SystemC part of the design is not required, then you can add flag `-kdb=sysc_blackbox_elab` to the VCS elaboration. This avoids the delay when Verdi is invoked for the first time but the SystemC models are visible in the *nTrace Instance* pane.

For example,

```
% syscan -kdb -cflags -g <the SystemC files>
% vlogan -kdb <the Verilog files>
% vcs -sysc <top_module_name> -kdb -kdb=sysc_blackbox_elab
  -debug_access+pp
% ./simv -ucli -do dump.tcl
% verdi -ssf novas.fsdb -dbdir simv.daidir
```

The `dump.tcl` file contains the following commands:

```
fsdbDumpvars
run
```

This option is available only when SystemC is instantiated with the Verilog/VHDL but not for SystemC-top (sc_main) topology. SystemC signals can still be dumped into FSDB file and displayed in the *nWave* window. However, the corresponding SystemC instances are missing in the hierarchy pane.

If you build `simv` with `-kdb=sysc_blackbox_elab` option and later need to make the SystemC visible in the *Instance* pane, then exit Verdi and perform the following steps:

```
% ./simv -ucli -do dump_and_invokeElab.tcl
% verdi -ssf novas.fsdb -dbdir simv.daidir
```

The `dump_and_invokeElab.tcl` file contains the following commands:

```
fsdbDumpvars
cbug::invokeElabcom
run
```

UFE Interactive Debug With `-kdb=sysc_blackbox_elab` Option

The `-kdb=sysc_blackbox_elab` flag can also be used for interactive debug.

This avoids the delay when Verdi is invoked for the first time but the SystemC models are not visible in the *Instance* pane.

For example,

```
% syscan -kdb -cflags -g <the SystemC files>
% vlogan -kdb <the Verilog files>
% vcs -sysc <top_module_name> -kdb -kdb=sysc_blackbox_elab
  -debug_access+all
% ./simv -verdi
```

If you build `simv` with `-kdb=sysc_blackbox_elab` option and later need to make the SystemC visible in the *Instance* pane, then exit Verdi and perform the following steps:

```
% ./simv -ucli -do dump_and_invokeElab.tcl
% ./simv -verdi
```

The `dump_and_invokeElab.tcl` file contains the following commands:

```
cbug::invokeElabcom
quit
```

Debugger Setup With IUS

To generate the information the Verdi platform requires, the FSDB dumper must be enabled and the following option must be added in the compilation command line:

-Wcxx,-g: Shows and gets the file and line information for C/C++/SystemC related files in the Verdi platform.

For example,

```
# Enables FSDB dumper
> setenv LD_LIBRARY_PATH
  $VERDI_HOME/share/PLI/IUS/$platform:$LD_LIBRARY_PATH
> irun -access +r -Wcxx,-g -tcl dump.tcl -top <top_module_name> <your
  design files>
> verdi -ridb default.ridb -ssf novas.fsdb
```

The `dump.tcl` file contains the following commands:

```
call fsdbDumpvars
run
exit
```

Note:

If a design contains HDL and SystemC code, it is necessary to add the following options **-scAllowSCFromPLI -ncsim_args,+scgafap+off** for appropriate time sync.

With Verdi 2014.12 version, when using IUS with SystemC dumping, ensure that the IUS library `libncscctlm2_sh.so` is linked into the simulation. This occurs automatically when you compile with `ncsc_run`. For example, if you use `ncsim`, then the library must be specified.

Set up a Directory for SystemC Database

You can specify a directory for SystemC database with the **-sc_dir** option if the **simv.daidir** directory is not writable or the SystemC hierarchy is changed. This directory is created so that the SystemC database can be reused when **simv.daidir** directory is not writable or the SystemC hierarchy is changed.

Note:

The SystemC hierarchy does not change in many scenarios.

When the design is imported, the RIDB/KDB file in the **sc_dir** has higher priority than the RIDB/KDB in the **simv.daidir** directory.

When the simulator is invoked and the SystemC hierarchy is different between the design and the simulator, then the RIDB file is dumped, the KDB database is regenerated in the **-sc_dir** directory, and then the design is loaded again. The SystemC database is updated in the directory and can be reused next time the directory is created.

Supported TCL Command

The following TCL command is supported for the **-sc_dir** directory:

debImport

Description

This command imports the design with the SystemC database in the specified **sc** directory.

Syntax

```
debImport -dbdir /remote/simv.daidir -sc_dir /local/scDir
```

Arguments

-sc_dir sc_directory

Specify the directory to dump SystemC database for SystemC or SystemC mixed design in the UFE flow.

Value Returned

Number of windows closed by this command.

Example

```
debImport -dbdir /remote/simv.daidir -sc_dir /local/scDir
```

Simulator Setup With Questasim

To generate the information, Verdi platform requires the FSDB dumper to be enabled and the following option to be added in the compilation command line:

1. Create the library by executing the following command:

```
> vlib work
```

2. Perform the common compilation on the Verilog design by executing the following command:

```
> vlog +define+NEW_PLI=1 -sv *.v
```

3. Compile the *.cpp codes.

```
>sccom -g *.cpp
```

4. Link the **libMTISC.so** file to the **systemc.so** file.

```
>sccom -link $VERDI_HOME/share/PLI/MODELSIM/$PLATFORM/libMTISC.so
```

This library is required for the new SystemC dumping.

5. Load the Verdi dumper library for FSDB dumping when running the simulation.

```
>vsim -pli $VERDI_HOME/share/PLI/MODELSIM/$PLATFORM/novas_fli.so
sc_main -c -do "fsdbDumpfile sc.fsdb; fsdbDumpvars 0; run 1000 ns;
exit"
```

You can invoke the FSDB commands at the **vsim** prompt.

For example,

```
VSIM 1>fsdbDumpfile test.fsdb
VSIM 2>fsdbDumpvars 0 system (Dump all signals from the system region)
```

6. Invoke Verdi

```
>verdi -ridb default.ridb -ssf novas.fsdb
```

Generate FSDB

The Verdi platform provides several dumping commands that can be used to generate the FSDB files. For dumping SystemC data into the FSDB file, the main dumping command is **\$fsdbDumpvars**.

The following table provides the usage of the **\$fsdbDumpvars** dumping command with different options to dump different SystemC data types. If you do not want to dump values, this step can be skipped.

Usage	Dump Type
\$fsdbDumpvars	SystemC signal, and ports (sc_in, ...)
\$fsdbDumpvars("+sc_plain")	Default items and plain members
\$fsdbDumpvars("+sc_struct")	Default items and variables of user-defined structs/classes/union/enum
\$fsdbDumpvars("+sc_fifo")	Default items and SystemC sc_fifo and tlm_fifo
\$fsdbDumpvars("+all")	Dump all supported items (including plain members). For SystemC dumping, equal to the \$fsdbDumpvars("+sc_struct", "+sc_plain", "+sc_fifo") usage

Note:

The FSDB dumping commands are only supported when called in the HDL code or on the Tcl command line. The dumping commands are not supported in the SystemC source code.

See the *Linking Novas Files with Simulators* and *Enabling FSDB Dumping manual* for a description of all FSDB dumping commands.

The following sections describe how to dump **sc_fifo** and **tlm_fifo** to an FSDB file in detail.

Dump FIFO to FSDB

You can dump **sc_fifo** and **tlm_fifo** to an FSDB file using the **\$fsdbDumpvars** command. For flexible usage (for example, for all simulations, a specific simulation, or a specific task), the following methods are also provided to achieve the same effect:

- Environment Variable: Set the **FSDB_SC_FIFO** or **FSDB_ALL** environment variables before executing simulations. For example:

```
> setenv FSDB_SC_FIFO 1
```

- Simulator Option: Add the **+fsdb+sc_fifo** or **+fsdb+all** options on the simulation command line. For example:

```
> ./simv +fsdb+sc_fifo
```

- Dumping Command: Add the **+sc_fifo** or **+all** options in the dumping task. For example:

```
> $fsdbDumpVars("+sc_fifo");
```

Dump Analog Signals in MSV

You can dump analog signals in MSV using the dumping command **fsdbDumpvars**.

To dump analog signals in MSV, perform the following steps:

1. Generate FSDB file for dumping SystemC data into the FSDB file, using the dumping command **fsdbDumpvars** as follows:

```
$fsdbDumpvars("+msv+v")
```

2. Invoke Verdi using the following command:

```
verdi -ssf novas.fsdb -msv
```

Setting the FIFO Size Environment Variable

You can use the `NOVAS_FSDB_MAX_FIFO_SIZE` environment variable to specify the value for the maximum size of the **sc_fifo** when the dumping option for **sc_fifo** is set as follows:

```
> setenv NOVAS_FSDB_MAX_FIFO_SIZE <num>
```

The `num` should be a positive integer.

For example,

```
> setenv NOVAS_FSDB_MAX_FIFO_SIZE 2000
```

If the size of the **sc_fifo** exceeds the specified value, the data numbers exceeding the value are not dumped. If the size is less than or equal to the specified value, the sized number of the data are dumped. The default maximum value of the environment variable is *100,000*.

Create the RIDB File

You can skip this section if you use the VCS with the UFE flow.

The RIDB file contains the SystemC hierarchy that Verdi needs to show the mixed SystemC/HDL instance tree. The RIDB file is automatically generated in the UFE flow (**-kdb**), is named "**default.ridb**" and stored within the "**daidir**" directory of the simulator.

If you not using the UFE flow, then it may be necessary to trigger creating the RIDB file with an explicit command.

- Use the `FSDB_DUMP_RIDB` environment variable or the **+fsdb+dump_ridb** runtime option to generate the RIDB file without modifying and re-compiling the design:

When any FSDB dumping command is used (For example, **\$fsdbDumpvars()** is called in the HDL code or **fsdbDumpvars** is called in the TCL command line), then Verdi platform can automatically dump the RIDB file during simulation. You must add the **+fsdb+dump_ridb** runtime option or set the `FSDB_DUMP_RIDB` environment variable during the simulation.

For example:

```
> ./simv +fsdb+dump_ridb
```

Or

```
> setenv FSDB_DUMP_RIDB 1  
> ncsim [runtime_options]
```

The default name of the dumped RIDB file is **default.ridb**. The **+fsdb+ridb_file** runtime option or the `FSDB_RIDB_FILE` environment variable can be used to assign a different RIDB file name.

For example:

```
> irun +fsdb+dump_ridb +fsdb+ridb_file=work.ridb [compile_options]
[runtime_options]
```

Or

```
> setenv FSDB_DUMP_RIDB 1
> setenv FSDB_RIDB_FILE "work.ridb"
> ./simv
```

- Use the dumping command or task to generate the RIDB file:

Directly use the **\$ridbDump(<file_name>)** dumping task in the HDL code or use the **ridbDump <file_name>** dumping command in the Tcl command line to dump the RIDB file. This usage is frequently applied in the Interactive Simulation Debug mode debugging (refer to the [Interactive Simulation Debug](#) chapter for details).

For example:

```
> $ridbDump(); //Call the task in Verilog Code
```

The default name of the dumped RIDB file is **default.ridb**. Optionally, you can assign the file name for the RIDB file.

For example:

```
> $ridbDump("work.ridb")
```

When the data type of a SystemC signal, port, or plain member is class or struct, the data members are dumped to RIDB file also. The data member of user defined class or struct can also be dumped.

Note:

If you want to generate only the RIDB file, you can use one of the dumping methods without running the simulation completely. For example, start a simulation in the Interactive mode, call the **ridbDump** task, and exit the simulation immediately.

2

Support Scopes

The Verdi platform provides the following features for SystemC debugging in the main windows (refer to [SystemC Debug Overview](#)):

- Displays SystemC/mixed design and hierarchy in the *Instance and Declaration* tabs of the *Hierarchy Browser* pane (refer to [Hierarchy Browser View](#)).
- Displays SystemC/mixed design and hierarchy in the *Instance and Declaration* tabs of the *Hierarchy Browser* pane (refer to [Signal List View](#)).
- Displays Signal, port and plain members of a SystemC instance in the *Signal List* pane (refer to [Signal List View](#)).
- Displays the C/C++ source file in plain text with syntax colors in the source code pane (refer to [Source Code View](#)).
- SystemC file or module can be found using the **Find Scope** command.
- SystemC signal, instance, or instance port can be found using the **Find Signal/Instance/Instport** command.

Note:

Objects cannot be dragged/selected/annotated in the source code pane.

Figure 1 SystemC Debug Overview

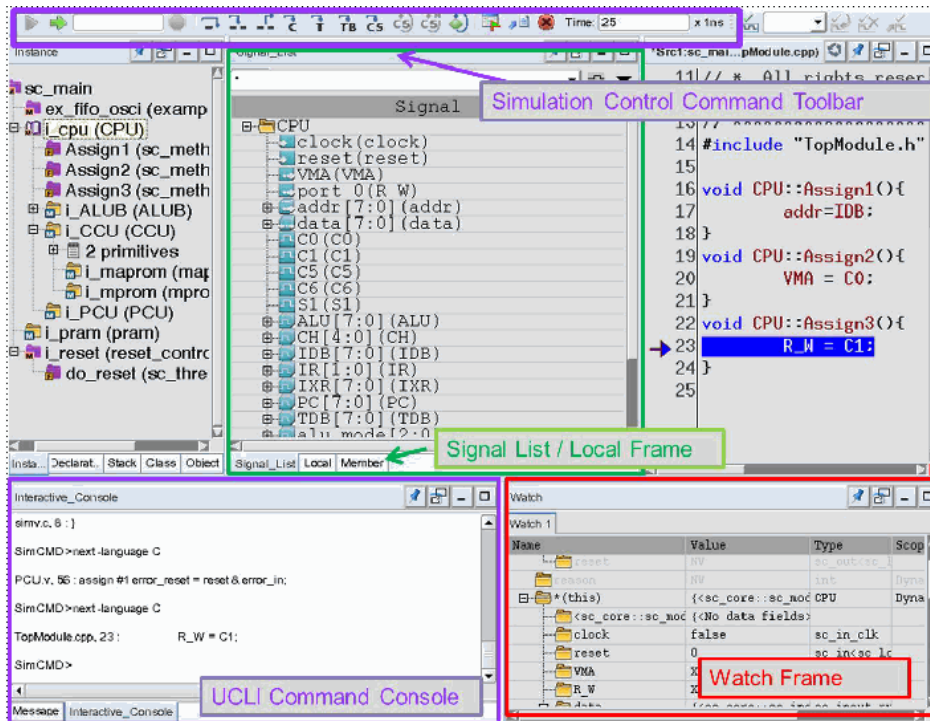


Figure 2 Hierarchy Browser View

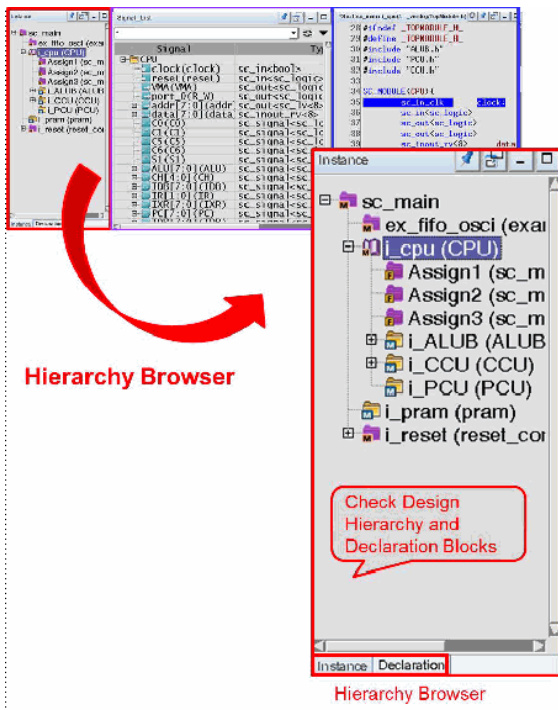


Figure 3 Signal List View

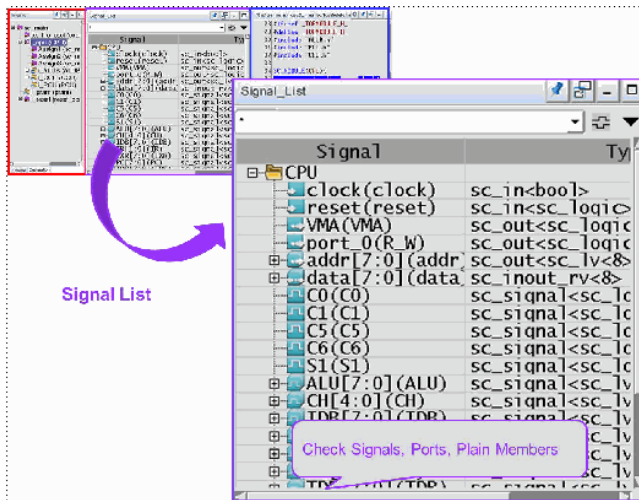
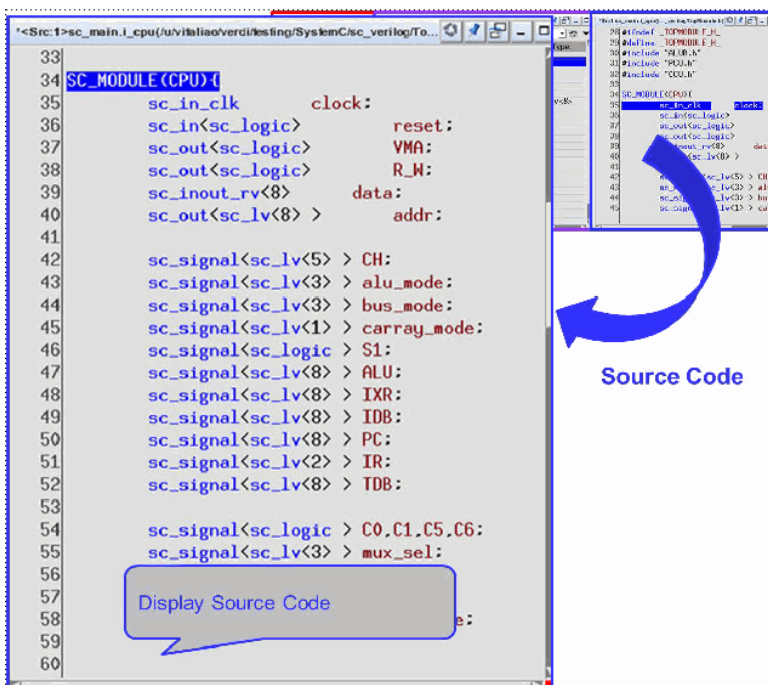


Figure 4 Source Code View

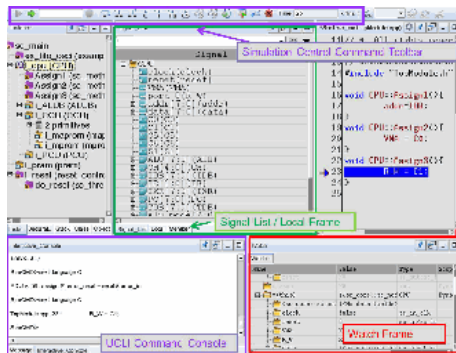


When entering the *Interactive Debug* mode, the simulation toolbar, menu, and interactive tabs are displayed (refer to [SystemC Interactive Debug Overview](#)). You can perform the following:

- Enable or disable C/C++ debugging.
- Check External DirectC/DPI/PLI functions, set breakpoints, and display the source.

- Set, delete, enable, and disable breakpoints in SystemC or any other C/C++ source files.
- Run or Continue the simulation.
- Step into or Step out of a function.
- Next in any code or just Next in C/C++.
- Check the local variables and change the value in the *Local* tab.
- Add signals and variables to the *Watch* tab.
- Check call stack and the sequence in the *Stack* pane.
- Display the value of the variable when the cursor is placed over the variable in the source code pane.
- Execute simulator commands and see the simulation information in the *Interactive Console* pane.

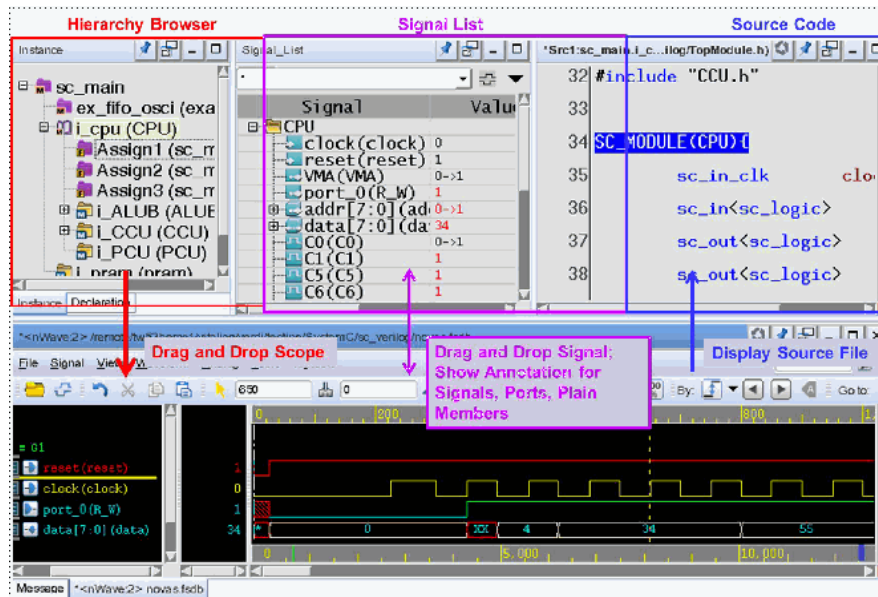
Figure 5 SystemC Interactive Debug Overview



Post processing debug is also supported (refer to [SystemC Post Processing Debug Overview](#)). After loading the design and FSDB files into the Verdi platform, you can perform the following:

- Get and show signals in the *nWave* window.
- Drag items from the *Hierarchy Browser* pane, *Signal List*, or *nWave* window and drop into the *Signal List* or *nWave* windows.
- Drop a scope or signal to the source code pane.
- Search signal value changes in the *Signal List* pane.

Figure 6 SystemC Post Processing Debug Overview



Supported Platforms and Simulator

The supported platforms are Red Hat Enterprise Linux (RHEL), SUSE, and Ubuntu.

All post-process debug features, such as *RIDB/nTrace*, *FSDB dumping/nWave* are available with different simulators:

- Synopsys VCS 2014.03 and above.
- Cadence IUS 13.1 and above.
- Mentor Modelsim 10.3 and above.

Interactive C/C++/SystemC debugging features are currently available only with VCS.

3

Usage

This chapter contains the following sections:

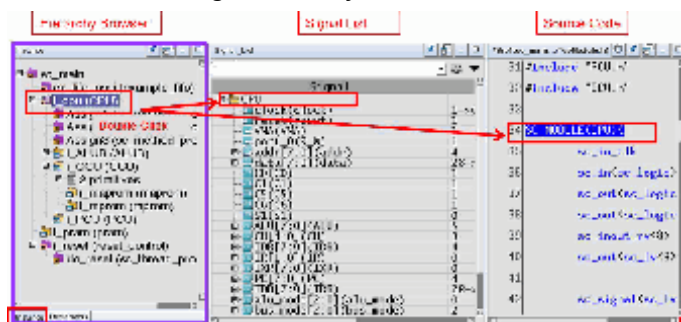
- [Main Window](#)
- [nSchema](#)
- [Showing FSDB](#)
- [Temporal Flow View](#)
- [nMemory](#)

Main Window

Hierarchy Browser

The hierarchy of the SystemC design can be viewed in the *Instance* tab. As shown in the following figure, you can see the overall hierarchy in the *Instance* tab. Double-click a node and the starting line of the block definition are shown and highlighted in the source code pane and the signals under the scope of the node are listed in the *Signal List* pane.

Figure 7 Showing Hierarchy in Instance Tab



The following table lists the naming rule and symbols for the node of the instance tree.

Node Object	Showing Name	Symbol
-------------	--------------	--------

Main Routine	sc_main	
Top Module	<module_name>	
Module Instance	<OSCI_name> (<module_name>)	
Process Instance	<process_name> (<process_type>)	

Signal List

SystemC signals and ports are displayed in the *Signal List* pane (refer to [Signal List Content](#)). The following variables can be also listed if the variables are dumped in the imported RIDB file and SystemC source files are compiled with debug info (gcc -g):

- Plain members
- Members of User-defined struct/class/union
- Struct/class data members

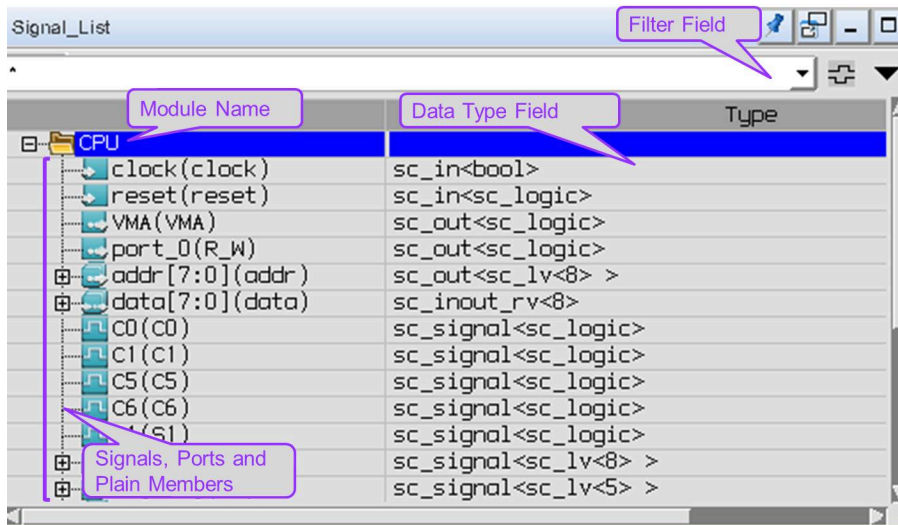
For all scopes, the head node is shown to indicate the scope. For top module or module instance, the signals are listed.

The following table lists the naming rule and children for the nodes of the signal list.

Head Note Type	Showing Name	Children
Main Routine	sc_main	Signals
Module Instance	<module_name>	Signals, ports, plain members (if they are dumped in the imported RIDB file)
Process Instance	<process_name>	None

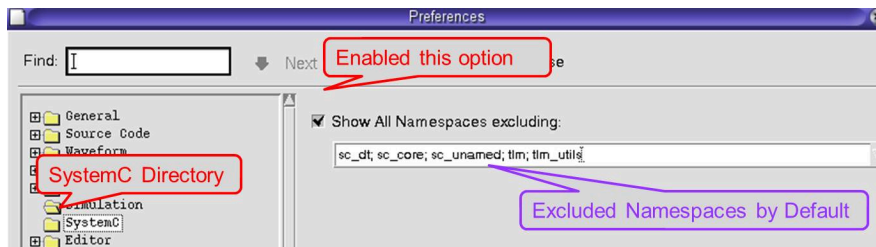
[Show All Namespaces](#) shows an example for a module instance with children in the *Signal List* pane.

Figure 8 Signal List Content



The **Type** column of the *Signal List* pane shows the data types without namespace, by default. You can use **Show All Namespaces excluding:** option in the **SystemC** folder of the *Preferences* form invoked by using the **Tools > Preferences** command to show all namespaces (as shown in [Show All Namespaces](#)). There are some default excluded namespaces when the **Show All Namespaces excluding:** option is enabled. You can modify them by entering specified namespaces or keep it blank to show all namespaces in the **Type** column.

Figure 9 Show All Namespaces



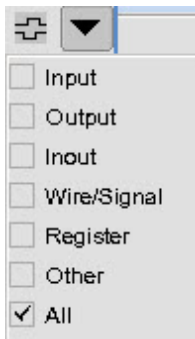
The presentation for the children of the module instance or top module is listed in the following table:

Child Data Type	Showing Name	Filter Types
SystemC signal	<OSCI name>[range] (<source name>)	Wire/Signal

SystemC Port	<OSCI name>[range] (<source name>)	Input / Output / Inout
Plain Member	<source name>	Other

The children in the *Signal List* pane can be filtered by clicking the  icon and selecting the filter type shown in the following figure from the filtered types menu.

Figure 10 Filter Types



A SystemC signal and data member can be dropped to the *Signal List* pane to locate it. You can select a signal in the *Signal List* pane to highlight it in the source code pane. When an FSDb file is loaded,

- You can double-click a signal to add it to the *nWave* window.
- You can enable the **Active Annotation** toggle command to show the corresponding value in the *Signal List* pane.

OSCI-name versus Source-name

All objects derived from the `sc_object` base class are visible in the SystemC design hierarchy and always have an “OSCI-name”. This name is either explicitly given in a constructor call or is assigned automatically by the SystemC kernel such as “port_0” or “signal_0”. Most objects are also members of a C++ class (usually an `SC_MODULE`) and have a “source-name” that is the name of the class member in the source code.

The OSCI and source names may be identical or different.

For example:

```
SC_MODULE(sctop) {
    sc_signal<int> A;
    sc_signal<int> B;
    sc_clock clock;
```

```
SC_CTOR(sctop) : A("A"), clock("clk") {}  
};
```

The presentation is as follows:

OSCI-name	Source-name	Display in Verdi Signal List
A	A	A(A)
signal_0	B	signal_0(B)
clk	clock	clk(clock)

The official name of SystemC objects is always the OSCI-name. The source-name is an optional piece of information that is used in the *Signal List* pane and similar places. Always use the OSCI-name in UCLI commands, for example, `get sctop.signal_0` and not `get sctop.B`.

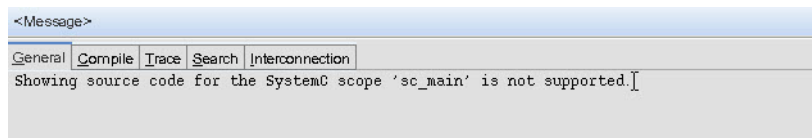
Source Code

For showing or getting the file and line information for object related files and `sc_main` source code in Verdi, you need to add the required options during compilation first as described in the [Setup and Requirements](#) section.

A warning message (as shown in [Warning Message for the Failing to Find the File](#)) is displayed in the *Message* pane for failing to show the source file in the following situations:

- The file is removed from the original directory.
- The file is not dumped into the RIDB file because it is not compiled with the **-g** option.
- In the Signal List window, selecting a signal of `sc_main` scope does not jump to the corresponding source code in the source code pane.

Figure 11 *Warning Message for the Failing to Find the File*

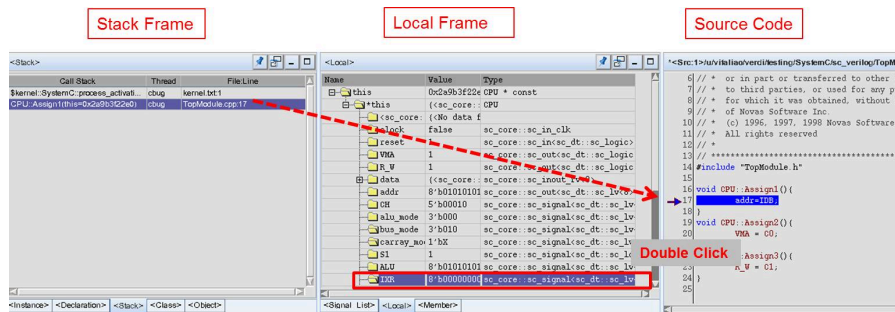


Verdi has limited capabilities for displaying C, C++, SystemC code in the Source Code pane:

- Dragging a signal from the Source Code to the Waveform window is not supported.
- Displaying variable/signal/port values directly within the source code (**Source...> Active Annotating, Parameter Annotation**) is not supported. Variable Value Tip is supported, see [Interactive Simulation Debug](#).
- Most features bound to the right-mouse button are not available for C, C++, and SystemC source code.

The following table lists the presentation in the source code pane when the SystemC/C/C++ object is selected in the hierarchy browser or *Signal List* panes.

Figure 12 Highlighted Source Code for Local Variable



C/C++ keywords and C/C++ identifiers are recognized. The C/C++ file is displayed in the source code pane with syntax colors.

SystemC/C/C++ Object	Operation and Display
Instance	The scope is changed to the module scope (that the instance is instantiated from) and the line of the module definition in the source file is highlighted.
Top Module	
Process	The scope is changed to the scope that the process belongs to and the line of the function definition in the source file is highlighted.
Signal	The scope is changed to the scope that the signal belongs to and the line of the signal declaration or the line of the related module definition in the source file is highlighted. The type definition is highlighted in case of user-defined structs/ports/unions.
Port	
Plain member	

Local variable in the <i>Local</i> and <i>Watch</i> frames (Interactive Debug Mode is enabled)	The scope is changed to the current scope of the simulation. The line of the executing call stack (shown in the <i>Stack</i> pane) in the source file is highlighted (refer to Highlighted Source Code for Local Variable).
--	--

nSchema

nSchema does not support SystemC scopes fully. This results in the inability to open a fully functional *nSchema* window for the SystemC scope. All the operations that try to invoke the inside net-list of the SystemC module are not made functional. An appropriate warning message is displayed as shown in the following figure:

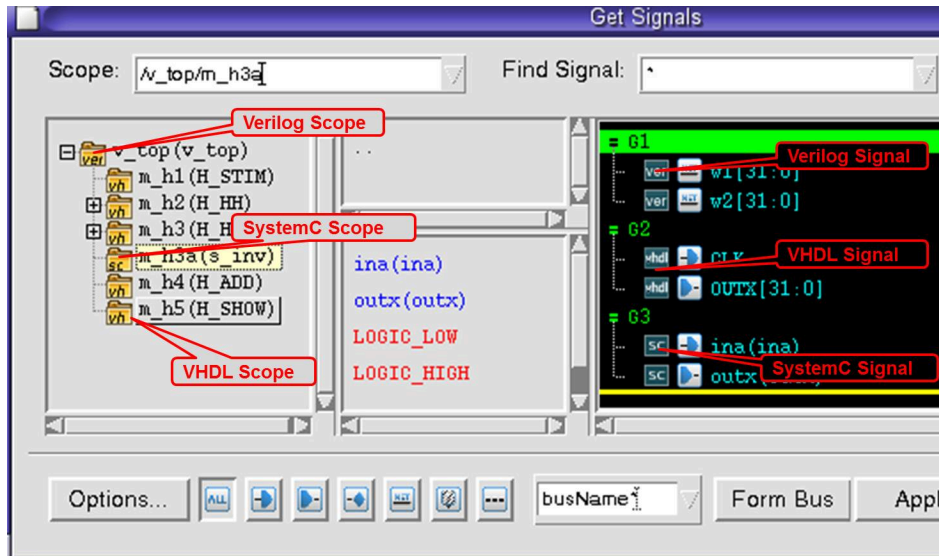
Figure 13 Warning Message on SystemC Scopes in *nSchema*



Showing FSDB

The dumped FSDB file for SystemC design can be displayed in the *nWave* window. Use the **Signal > Get Signals** command to open the *Get Signals* form after loading the FSDB file into the Verdi platform. As shown in [Symbols for SystemC](#), the  and  symbols indicate the SystemC scope and SystemC signal respectively. The name of the SystemC signals is shown as <OSCI name>(<source name>). You can drag a node from the **Instance** tab and drop it in the *nWave* window to add the SystemC signals belonging to the scope of the node. The SystemC signal can be dragged from the *nWave* window and dropped to other frames (for example, dropping to the source code pane jumps to the definition of the signal).

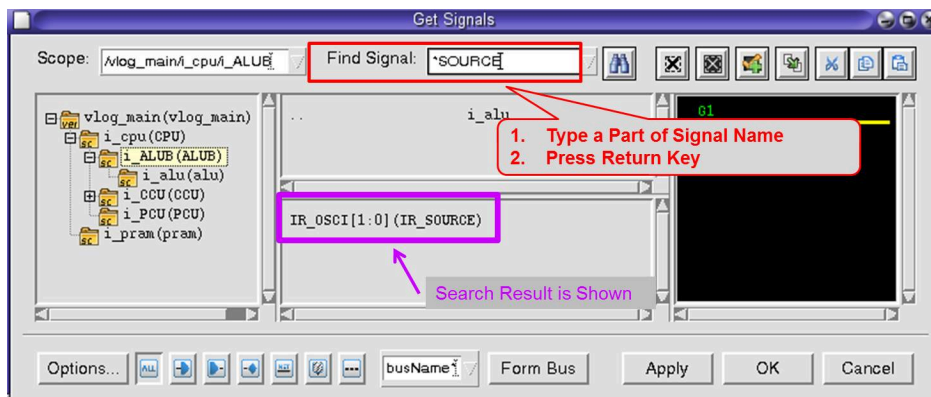
Figure 14 Symbols for SystemC



Search Signals

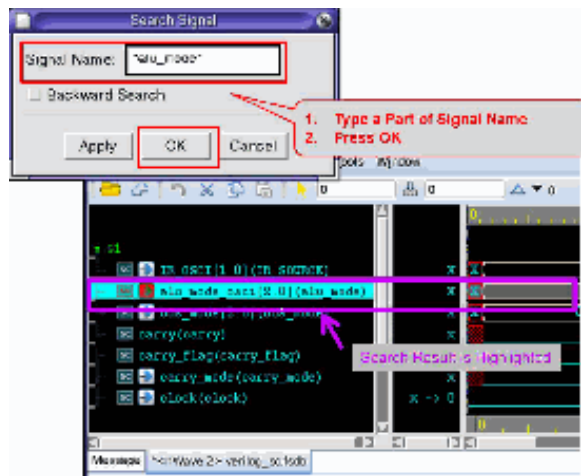
You can use the OSCI name or source name to find SystemC signals in the *nWave* window with a loaded FSDB file. Assumed there is a signal with a "IR_OSCI" OSCI name and "IR_SOURCE" source name, using either "IR_OSCI" or "IR_SOURCE" strings filters the signal. For example, type the "SOURCE" string in the **Find Signal** field of the *Get Signals* form and press the return key. As shown in [Search Signals](#), the "IR_OSCI[1:0] (IR_SOURCE)" signal is displayed.

Figure 15 Search Signals



After adding signals into the *nWave* window, you can use the **Signal > Find** command to search the signals in the *nWave* window (refer to [Find Signals](#)).

Figure 16 Find Signals



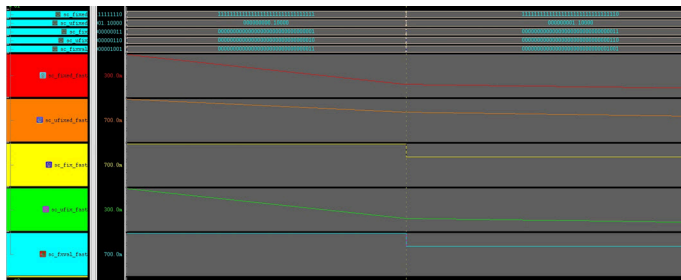
Special Appearance in nWave

The following two data types, fixed point and class, have special appearances in the waveform:

- Fixed Point:

As shown in [Fixed Point in nWave](#), the finite-precision types (`sc_fixed`, `sc_ufixed`, `sc_fix` and `sc_ufix`) are displayed as a digital bus with the binary radix. The limited-precision types (`sc_fixed_fast`, `sc_ufixed_fast`, `sc_fix_fast`, `sc_ufix_fast`, `sc_fxval`, and `sc_fxval_fast`) are displayed as an analog signal.

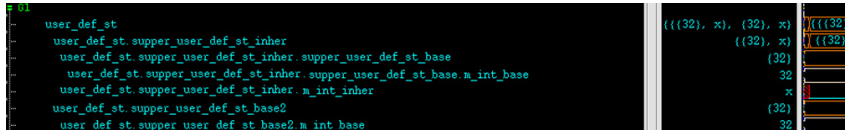
Figure 17 Fixed Point in nWave



- Class:

The classes are shown as struct signal. As shown in [Class in nWave](#), you can see the hierarchy for the inheritance relationship of classes.

Figure 18 Class in nWave

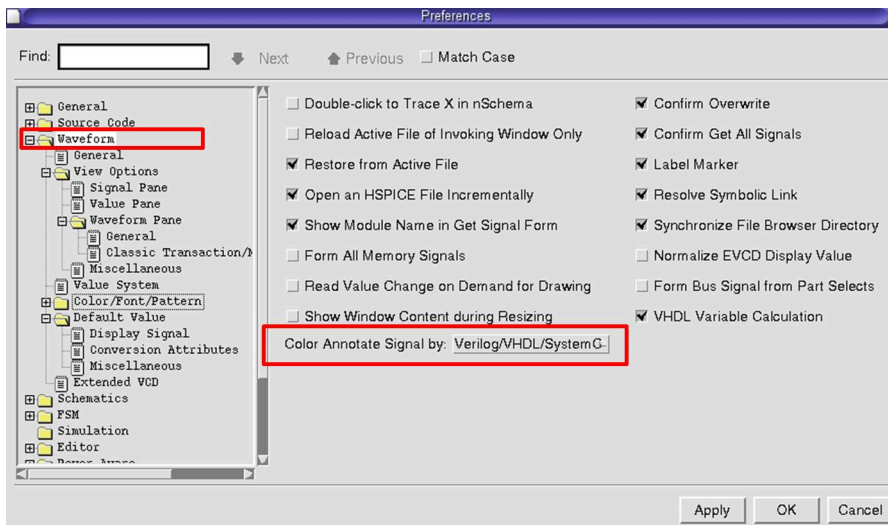


Highlight Signals

The SystemC signals can be highlighted in the *nWave* window using the following steps:

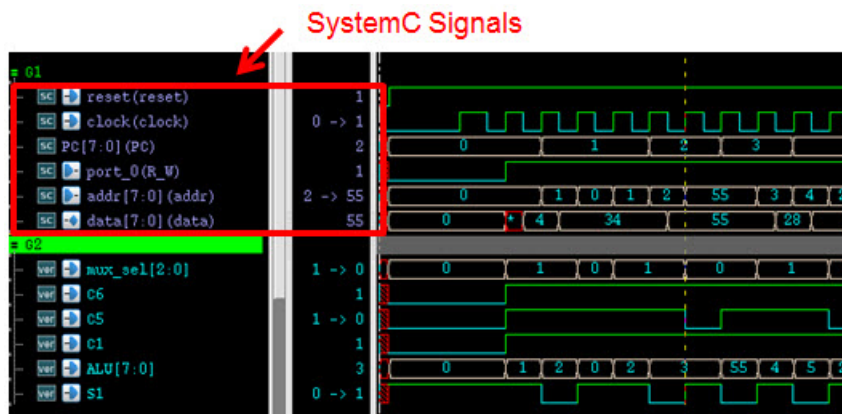
1. Invoke the **Tools > Preferences** command to open the *Preferences* form.
2. Select the **Color Annotate Signal by: Verilog/VHDL/SystemC** option in the **Waveform** page (refer to [Color Annotate Signal by Verilog/VHDL/SystemC in Preferences](#)).

Figure 19 Color Annotate Signal by Verilog/VHDL/SystemC in Preferences



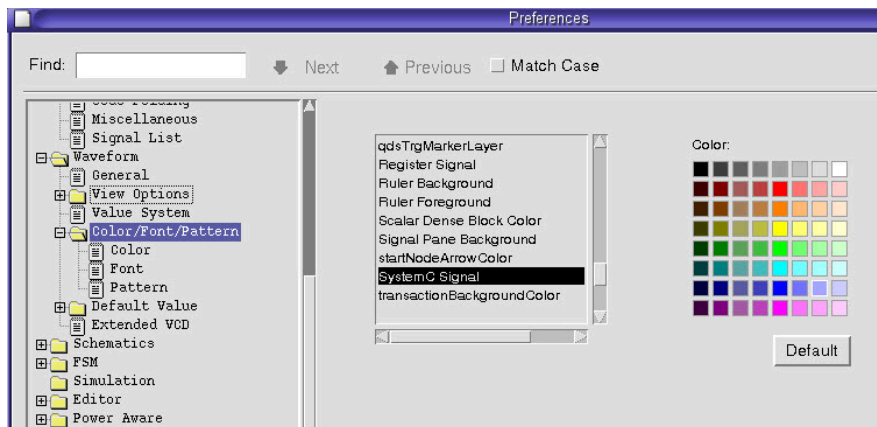
The SystemC signals are highlighted with a special color (refer to [Color Annotate Signal by Verilog/VHDL/SystemC](#)).

Figure 20 Color Annotate Signal by Verilog/VHDL/SystemC



The highlight color can be changed by selecting the **SystemC Signal** type and the desired color in the **Waveform > Color/Font/Pattern** page of the *Preferences* form (refer to [Select Specific Color for SystemC signals](#)).

Figure 21 Select Specific Color for SystemC signals



Temporal Flow View

SystemC signals cannot be traced in or added to the Temporal Flow View (TFV). When a SystemC boundary is reached while tracing RTL signals, tracing stops and the boundary is disconnected.

nMemory

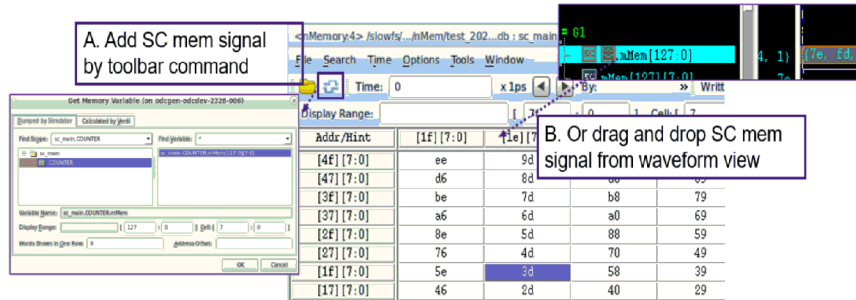
nMemory supports SystemC array when the array is constructed by the listed SystemC data-types.

The corresponding SystemC data type is as follows:

```
sc_bigint<128>, sc_biguint<128>, sc_bit, sc_bv<12>, sc_int<12>,
sc_int_base, sc_logic, sc_lv<12>, sc_signed, sc_uint<12>, sc_uint_base,
sc_unsigned
```

The following figure shows the SystemC memory signals in *nMemory* window:

Figure 22 Adding SystemC Array Signal into nMemory Window



The following are the limitations:

- As Temporal Flow View (TFV) does not support SystemC and selecting SystemC objects is not supported in source code, the following features are not supported:
 - Dump Memory Waveform to FSDB
 - Trace Memory Write
- As BA does not support SystemC and selecting SystemC objects is not supported in source code, the following features are not supported:
 - Calculated by Verdi
 - Memory Definition Table

4

Interactive Simulation Debug

The *Interactive Debug* mode is provided for SystemC interactive debugging. The following sections describe the flow of enabling *Interactive Debug* mode and its supported features.

Note:

The SystemC interactive debugging is provided with VCS but not with IUS or MTI.

Starting SystemC Interactive Debugging

After the RIDB file is prepared, you can invoke the Verdi platform by specifying the compiled design library (using the **-lib** option), enter the interactive debug mode (using the **-i** option), and specify the executable path (using the **-simBin** option) and simulation type (using the **-simType** option).

For example:

```
> ./simv -verdi
```

It can also be accomplished through the Verdi GUI as follows:

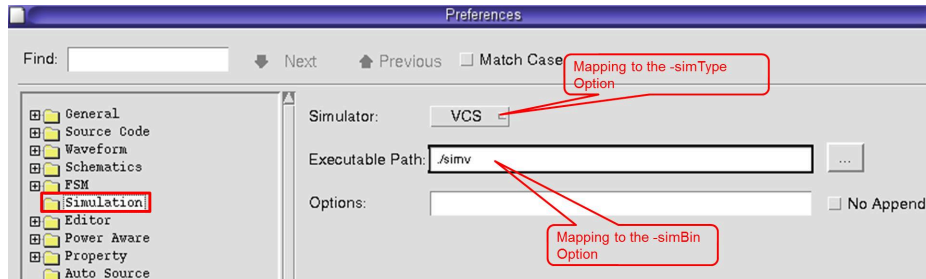
1. Invoke the Verdi platform.

```
> verdi &
```

2. Import the design using the **File > Import Design** command.
3. Invoke the *Preferences* form using the **Tools > Preferences** command to open and set the simulator option on the **Simulation** page (as shown in the [Configure the Simulation](#)).

4. Turn on *Interactive Debug* mode using the **Tools > Run Interactive Mode** command.

Figure 23 Configure the Simulation



Invoking Verdi Interactive Debug Mode With Unified Compiler Front-End

When the Verdi Knowledge Database (KDB) is generated using Unified Compiler Front-end, you can invoke Verdi with the KDB through one of the following simulator command-line options to perform interactive simulation debugging without other configurations:

- **-gui=verdi**
- **-verdi**

For example, when the following commands are used during the compilation and elaboration:

```
> vlogan -sysc -kdb <your Verilog files>
> syscan -cflags -g -kdb <your SystemC files>
> vcs -sysc <top_module_name> -debug_access+r -kdb -lca
```

You can use one of the following commands to invoke Verdi and enable the *Interactive Simulation Debug* mode:

```
> simv -gui=verdi
> simv -verdi
```

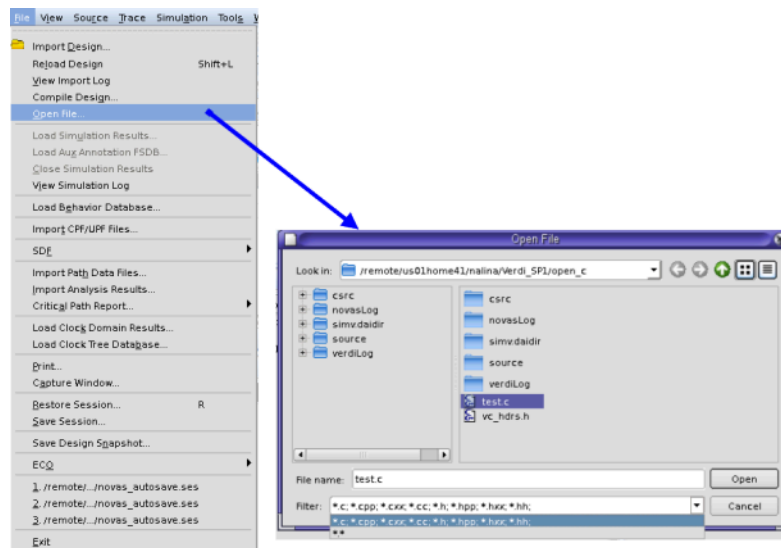
The Verdi platform is invoked in the *Interactive Simulation Debug* mode. If the `default.ridb` file does not exist in the `simv.daidir/` directory, then the `default.ridb` RIDB file is automatically generated and loaded into Verdi. If the `default.ridb` file already exists in the `simv.daidir/` directory, it is automatically loaded into Verdi.

Opening the C Source Code File

Verdi enables you open the external functions and set breakpoint in an opened C/C++/SystemC file when a design is imported. To open the C, C++, or SystemC code in the **Source** window, you can open the code file using the **File > Open File** command.

When you click **Open File** menu, the **Open File** window opens, as illustrated in the following figure:

Figure 24 Opening C Source Code File

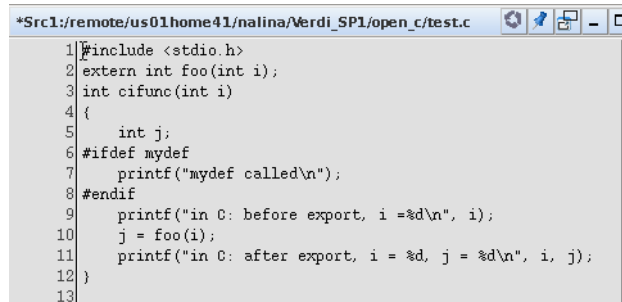


After you select the file type and the file you want to open, Verdi opens the file in the **Source** window.

If the design is not imported then all the files are displayed without the syntax color ([Source Code Displayed Without the Syntax Color](#)). However, even if the design is imported, the files are displayed without the syntax color under the following conditions:

- If the file is not part of the design.
- If the **C/C++ File Type** is not set in the *Preferences* form.

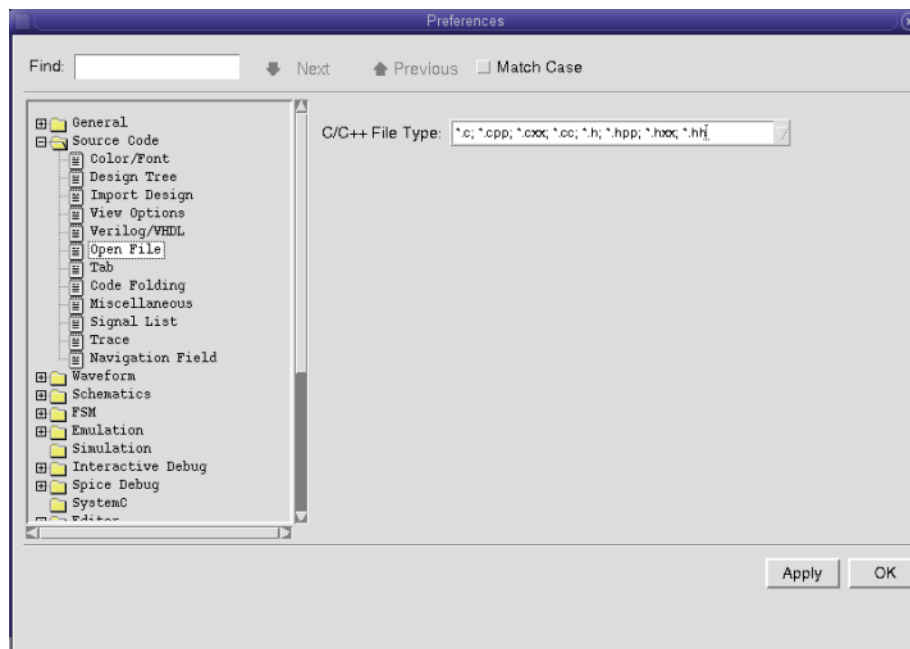
Figure 25 Source Code Displayed Without the Syntax Color



```
*Src1:/remote/us01home41/nalina/Verdi_SP1/open_c/test.c
1 #include <stdio.h>
2 extern int foo(int i);
3 int cifunc(int i)
4 {
5     int j;
6     #ifdef mydef
7         printf("mydef called\n");
8     #endif
9     printf("in C: before export, i = %d\n", i);
10    j = foo(i);
11    printf("in C: after export, i = %d, j = %d\n", i, j);
12 }
13
```

To set the preferences, select **Tools > Preferences > Source Code > Open File > C/C++ File Type**, as illustrated in the following figure:

Figure 26 Setting Source Code Preference



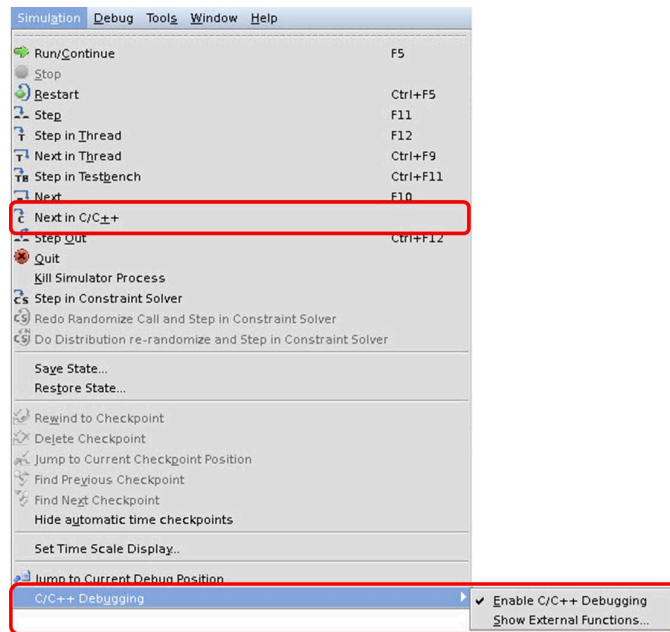
Limitation

When you open a SystemC file, the SystemC file is displayed without the scope. This is because Verdi cannot get the information of the scope from the SystemC file.

Enabling C/C++ Debugging in Interactive Debug Mode

For C/C++ debugging, the **Simulation > Next in C/C++** command (also in the simulation control command toolbar) and the **Simulation > C/C++ Debugging** commands are added, as illustrated in the following figure:

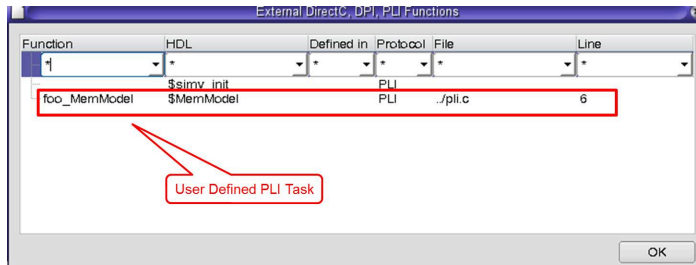
Figure 27 New C/C++ Debugging Commands



You can use the **Simulation > Next in C/C++** command or click the  icon in the toolbar to move the simulation to the next line in the C/C++ code.

Use the **Simulation > C/C++ Debugging > Show External Functions** command to show all the DirectC, DPI, and PLI functions in the *External DirectC, DPI, PLI Functions* form (refer to [External DirectC, DPI PLI Functions](#)). You can show the source code and set the breakpoint for external functions in the form using the **Show Source** and **Set Breakpoint** commands in the right-click context menu. The two commands are disabled if file/line number information of the function does not exist in the simulator.

Figure 28 External DirectC, DPI PLI Functions



The C/C++ debugging function is turned *off* by default. The function needs to be turned *on* to enable C/C++ debugging.

The C/C++ debugging function can be explicitly enabled with one of the following methods:

- Execute the simulator command in the *Interactive Console*:

```
SimCMD> cbug
```

- Select the **Simulation > C/C++ Debugging > Enable** toggle command.

In addition, C/C++ debugging is enabled automatically with the following actions if it is not disabled with the **cbug config** simulator command.

- Set a breakpoint in C code.
- Use the **Simulation > Next in C/C++** command.
- Use the **Simulation > C/C++ Debugging > Show External Functions** command.

C/C++ debugging fails to be enabled if there are checkpoint(s). A warning message is displayed in the *Interactive Console* window as follows:

Figure 29 Warning Message on the Interactive Console

```
Error: [UCLI-138] Checkpoint/CBug feature not supported.  
If there are checkpoints present then CBug will not be supported, or if CBug is attached, checkpointing will not be supported, in this release.  
Please refer to UCLI user guide for more documentation on supported use model of checkpoint command.
```

You can disable C/C++ debugging with one of the following methods:

- Execute the simulator command in *Interactive Console* frame:

```
SimCMD> cbug -detach
```

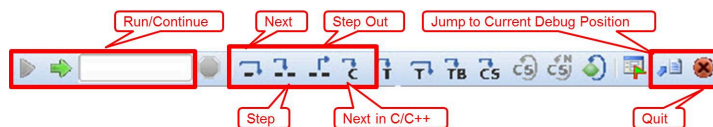
- Uncheck the **Simulation > C/C++ Debugging > Enable** toggle command.

Simulation Control Commands

The existing commands are also supported (click the icons shown in [Simulation Control Commands Toolbar](#) or you can select the command from the **Simulation** menu) by the simulator for both HDL and C/C++ debugging:

- **Run/Continue:** Simulation stops at a breakpoint setting in the HDL code or C code.
- **Restart:** Restart the simulation.
- **Step:** Simulation steps within the HDL code, C code, and cross step from HDL code to C function.
- **Next:** Simulation goes to the next executed statement within the HDL or the C code.
- **Next in C/C++:** Simulation continues through the HDL code and stops in the next C/C++/SystemC statement to be executed. If an HDL breakpoint is reached before the next C/C++/SystemC statement, then the simulation stops at this breakpoint. Same as UCLI command 'next -lang C'.
- **Step Out:** Simulation steps out within the HDL code, C code, and step out to HDL code from C function. Same as UCLI command 'next -end'.
- **Quit:** Quit the simulation.
- **Kill Simulator Process:** Kill the simulator process.
- **Jump to Current Debug Position:** The cursor jumps to the current debug line in the source code pane.

Figure 30 Simulation Control Commands Toolbar



When the simulation is stopped in the SystemC/C/C++ code, the stop information is shown in the status bar, as illustrated in the following figure:

Figure 31 Stop Status



If the simulation is stopped in a valid SystemC scope, the message is “*stopped: <scope_name>*”. As shown in the following figure, the simulation is stopped in the `sysc_top.sysclv1` scope. This is usually the scope name of a SystemC process. In case of TLM 2.0, the scope shows the target module. If the simulation is stopped in a C/C++

+ function with unknown scope (for example, stopped in a PLI function from HDL), the following message is displayed: `stopped: C/C++`.

For SystemC/C/C++ files, the following types of breakpoints are supported:

- Set File/Line breakpoint
- Set Function breakpoint
- Set Time breakpoint

C-Source Debugger (CBug) Functions

You also can use the C-source debugger (CBug) functions by executing UCLI commands in the *Interactive_Console* window to debug the design. The following sections introduce the features and usages of the UCLI commands.

next -hdl

When the current scope of the simulation is a task/function or in C/C++/SystemC, you can use the **next -hdl** command to advance the simulation until you exit the C domain and are back in the HDL domain. If a breakpoint in the C domain is reached first, then the simulation stops at this breakpoint.

Cross-step into SC Processes

CBug offers specific support for stepping between SystemC or HDL processes.

- When you leave a function that defines a `SC_METHOD` process, then a **Step** or **Next** command automatically stops in the next SystemC or HDL process, which comes next.
- Similarly, when you use the **Step** or **Next** command over a 'wait' statement that belongs to an `SC_THREAD` process, then you can stop in the next SystemC or HDL process, which comes next.
- Doing a **Step** or **Next** in Verilog or VHDL automatically stops in a SystemC process if that process occurs to be the next SystemC or HDL process to be executed.

That is, performing **Step** or **Next** repeatedly follows the flow of SystemC and HDL processes in the exact order in which the simulator executes them.

Step Through SystemC

This is not a separate UCLI command but a CBug configuration that affects how a **Step** works.

- A **Step** command may be required that steps through to the next line in the user code and does not step into SystemC kernel functions such as `"sc_port::read()"`.
- **Configuration:** Use the following command to enable the step-through feature: **cbug::config step_through on**. The feature is disabled by default but prints a message after reaching the first situation where a stop-through might have made sense.
- **Recovering from Error Conditions:** In some cases, it is possible that an automatic step-through does not quickly stop at a statement, but triggers another step-through, followed by another step-through, and so on. In this case, it may make the Verdi platform appear unresponsive, but you may not be aware that the step-through is still active. CBug recognizes this situation and takes action. If a step-through does not stop on its own after 10 consecutive iterations of an internal finish or step, then CBug can stop the chain of internal finish or step sequences on its own, and report a warning that states the automatic step-through is aborted and provides instructions to disable it.

Cross-Step between SV and PLI/DPI

When you step through HDL code and come to a call of a user-provided C function, such as a PLI function like `$myprintf`, then the **Next** command steps over this function. However, the **Step** command steps into the C source code of this function (if CBug is already attached).

Subsequent **Step/Next** commands walk through the C function and finally return you to the HDL source; therefore, seamless stepping from HDL → C → HDL is possible.

Cross-stepping is supported only for functions that meet the following criteria:

- **import_DPI** functions
- PLI functions that are statically registered through a TAB file. Cross-stepping into the **call** function is supported, cross-stepping into the **misc** or **check** function is not supported.
- **DirectC** functions

Cross-stepping into other Verilog PLI functions is not supported. However, an explicit breakpoint can be set in these unsupported functions to achieve the same effect.

Direct GDB Commands

You can send certain commands directly to the underlying gdb through the UCLI command **cbug::gdb**. The command is executed immediately and the UCLI command returns the response from gdb.

The command is as follows:

```
SimCMD>cbug::gdb <gdb-cmd>
```

gdb-cmd is an arbitrary command accepted by gdb including an arbitrary number of arguments, for example **info sources**. When the **cbug::gdb** command is used in the *Interactive_Console* window, CBug is automatically attached and the *<gdb-cmd>* arbitrary command is sent to gdb. The result of the command is returned from gdb in a similar manner as a Tcl routine and may have one or multiple lines.

The routine returns successfully in most cases, even if gdb itself gives an error response. The routine gives a Tcl error response only when **gdb-cmd** has the wrong format, for example when it is empty.

For example:

```
SimCMD> cbug::gdb info sources
```

The returned result is:

Source files for which symbols have been read in:

```
../pythag.c, rmapats.c, ctype-info.c, C-ctype.c, C_name.c,  
...
```

Source files for which symbols will be read in on demand:

```
../../ncurses/tinfo/lib_tparm.c, ../../ncurses/tty/lib_twait.c,  
...
```

Some gdb commands are not allowed and trigger an error message from CBug. These are commands that change the state of gdb or simv (for example, commands **kill**, **restart**, **break**).

Local Tab

The local C/C++ variables for the current scope are shown in the **Local** tab, including:

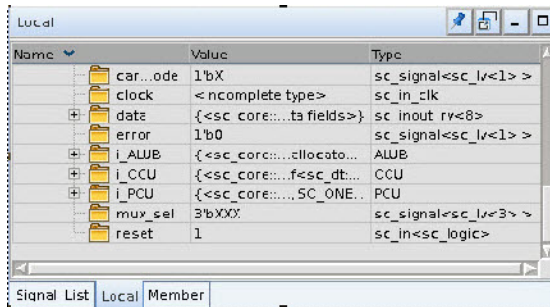
- Function arguments
- “this” pointer if the function is a member function
- Local variables in current scope

As shown in [Local Variable Shown in the Local Tab](#), the **Local** tab shows the variables with the name, value, and data type. The variables are collapsed by default.

Scalar variables which are native SystemC types such as `sc_lv<3>` or `sc_in<int>` are shown in a pretty-print form. For example, variable `mux_sel` in [Local Variable Shown in the Local Tab](#) has the native SystemC type `sc_signal<sc_lv<3>>` and the value is shown as

3'bXXX. If you want to bypass pretty-printing and instead see the raw response from GDB, then type the **cbug::gdb print mux_sel** command in the console. Expand class/struct/unions objects to see all members also in pretty-print form.

Figure 32 Local Variable Shown in the Local Tab



When the following situations are met, the local variables of the current stack frame are updated:

- Simulator stops in C/C++ code.
- The current stack frame of C/C++ stack (shown in the *Stack* window) is changed.
- The local variable value is changed (refer to the [Change Value](#) section for details)

Note:

Objects cannot be dropped (added) to the **Local** tab.

Pretty-printing does not support nested typedefs and displays the value in raw GDB format.

Change Value

The **Change Value** command in the right-click menu is provided to change the value of the selected variable in the **Local** tab.

Figure 33 Force Signal Value



As shown in [Force Signal Value](#), enter a new value in the *Force Signal Value* form and the changed value is overwritten by a subsequent driver when the **Deposit** option is enabled and is frozen when the **Freeze** option is enabled.

When the value is changed, Verdi calls the simulator commands to change the value and show the simulator result in the *Interactive Console* window. If the value is changed successfully, the variable value is changed in the **Local** tab; if not, there are no further actions.

You also can use the **change** UCLI command to change the values of SystemC and local C objects. The following command describes the usage and restrictions.

CBug supports changing the value of C variables and the SystemC `sc_signal`. For example:

```
SimCMD> change my_var 42
SimCMD> top.inst0.signal_0 "1100ZZZZ"
```

SystemC Variables

Only SystemC objects of types `sc_signal` and `sc_buffer` can be changed. Changing the value of ports, `sc_fifo` or any other SystemC object is not supported. SystemC objects must be addressed by their full hierarchical path name or by a name relative to the current scope.

For example:

```
SimCMD> scope top.inst1.sub_inst
SimCMD> change top.inst0.signal_0 42 // correct usage
SimCMD> change signal_0 42 // wrong usage, local path is not supported for SystemC
SimCMD> scope top.inst0
SimCMD> change signal_0 43 //correct, scope and local
```

Only `sc_signals<T>` / `sc_buffer<T>` of certain data types T are supported:

- All kinds of ANSI integer types, for example, `int`, `long long`, `unsigned char`, also `bool`
- Native SystemC bit-vector types: `sc_lv`, `sc_bv`, `sc_int`, `sc_uint`, `sc_bigint`, `sc_biguint`

User-defined data types are not supported.

All SystemC `sc_signals` follow the "Current value versus Next value" policy. Any value change, either from the user C++ code or a **change** command, modifies the next-value only but not the current-value. The current-value is updated only with the next SystemC delta cycle. The effect of a successful **change** command may therefore not be directly visible. If you query the value with the **get** UCLI command, the next-value is displayed

because the **get** command retrieves the next-value, not the current-value for `sc_signals`. However, accessing the `sc_signal` with a `read()` inside the user C++ code, shows the current-value until the next SystemC delta cycle occurs. CBug prints a message explaining that the assignment is delayed until the next delta cycle.

A change may compete with other accesses from inside the user C++ code. If the signal is first modified by a **change** command but later a `write()` happens within the same delta-cycle, then the `write()` wins and cancels the effect of the earlier **change** command. A permanent change also known as the *force -freeze* is not supported.

The format of the value specified with the **change** command is defined by the corresponding SystemC data type of T. ANSI integer types expect decimal literals; native SystemC bitvector types accept integer literal and bit-string literals.

For example, if the SystemC module `top.inst_0` contains the following:

```
sc_signal<int> sig_int
sc_signal<sc_int<8>> sig_sc_int
sc_signal<sc_lv<40>> sig_sc_lv
```

then, these change commands are acceptable:

```
SimCMD> change top.inst_0.sig_int 42          //assign decimal 42
SimCMD> change top.inst_0.sig_sc_int 0d015     //assign decimal 15
SimCMD> change top.inst_0.sig_sc_int 0b0111zzXX //assign bin value
SimCMD> change top.inst_0.sig_sc_int 0x0ffab   //assign hex value
SimCMD> change top.inst_0.sig_sc_int 15        //assign decimal 15
SimCMD> change top.inst_0.sig_sc_int -15       //assign decimal -15
SimCMD> change top.inst_0.sig_sc_lv 0d015      //assign decimal 15
SimCMD> change top.inst_0.sig_sc_lv -0d015     //assign decimal -15
SimCMD> change top.inst_0.sig_sc_lv 0b0111zzXX //assign bin value
SimCMD> change top.inst_0.sig_sc_lv 0x0ffab    //assign hex value
SimCMD> change top.inst_0.sig_sc_lv 0011zzXX   //assign bin value
```

Local C Variables:

Local C variables are variables that are visible within the current C/C++ stack window. This is usually the location where the simulation is stopped. However, the window can be changed by double-clicking a specific window in the **Stack** window or with the UCLI **'stack -up'** or **'stack -down'** commands. Local C variables are formal arguments of functions/methods, local variables declared inside a function/method, member variables visible in

the current member function, and global C variables. All these are referred to as "local C variables".

Support for changing local C variables is quite different from SystemC objects. The following restrictions apply to changing local C variables:

- CBug must be attached.
- Only simple ANSI types can be changed: bool, all kinds of integers (for example, signed char, int, long long), 'char*' and pointers. Arrays of these types are also acceptable if only a single element is changed.
- The format of the value is defined by gdb, for example, 42, 0x2a, 'a', "this is a test".
- SystemC types are not supported. For example, no support for sc_int, sc_lv.
- STL types such as std::string, std::vector, and so on are not supported.
- Using the full path name (for example, top.inst_0.my_int) is not supported, only local names (for example, my_int or this->my_int)

For example:

```
void G(int I){
    char* S = strdup("abcdefg");
    ...
}
void F(){
    int I=42;
    G(100);
    ...
}
```

Assume that the simulation stops in function G in line 103.

```
SimCMD> change I 102      // change formal arg I from G defined in line 100

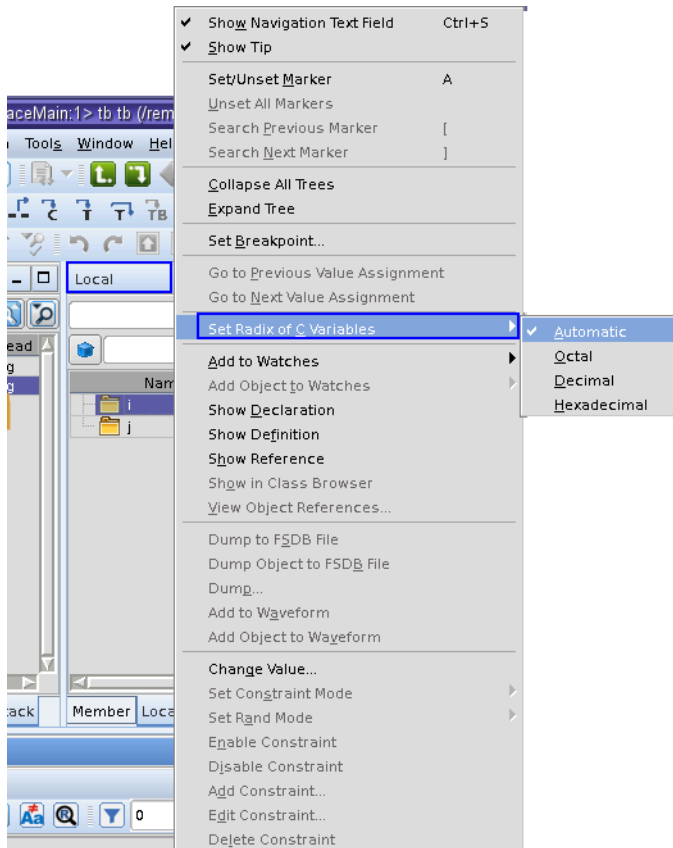
SimCMD> change I 0xFF
SimCMD> change S "hi j kl"
SimCMD> change {S[1]} 'I'
SimCMD> scope -up

SimCMD> change I 42      // change variable I from F defined in line 108
```

Setting the Radix of C Variables

You can change the default radix in which the values of C/C++/SystemC variables are displayed in the **Local** tab and **Watch** tab. Open the context menu with the right-click menu and select the desired radix from the **Set Radix of C Variables** option, as shown in the following figure. The radix setting applies to integer types and native SystemC bit-vectors.

Figure 34 Setting the Radix of C Variables



Filter and Search Functions

You can filter or search for the current tree in the **Navigation** fields. Display the fields by selecting the **Show Navigation Text Field** command of the right-click menu in the **Local** tab.

Figure 35 Filter and Search Function



Input a filtering string to the field and click **Filter** to filter the tree. If no matching object name with the specified string exists the tree node is filtered out. To search the tree, input a string to the field and then click the  icon to search backward or press the  icon to search forward.

Note:

The **Filter Type** is disabled in the **Local** tab if the simulator stops in a C domain.

Watch Tab

The C/C++ local variables or SystemC signal/port/plain members can be added to the **Watch** tab using the following methods:

- Use the **Add to Watch Tab** command of the right-click menu for the selected variable in the **Local** tab or drag the variable in the **Local** tab and drop to the **Watch** tab
- Drag a signal/port/plain member from the *Signal List* pane to the **Watch** tab

The **Watch** tab shows the variables or signals with the name, value, and data type. The signal scope or the current call stack pane is also shown for a signal or local variable. The local variable in the **Watch** tab is disabled (grayed out) if it is not in the current stack pane.

The signal values in the **Watch** tab is updated when the simulation time is changed and the local variable value is updated in the following situations:

- The simulator stops.
- The current stack frame of C/C++ stack (shown in the *Stack* pane) is changed.
- The local variable value is changed (refer to the [Change Value](#) section for details)

You can double-click a local variable to see the call stack pane and double-click a signal/port/plain member to show the source code in the source code pane.

You can also change the radix of the C variable using the right-click option **Set Radix of C Variable** (for more information, see section [Setting the Radix of C Variables](#)).

Filter and Search Functions

You also can filter or search for the current tree in the **Navigation** fields in the **Watch** tab. Refer to the [Filter and Search Function](#) section for details. For the **Watch** tab, the filter types are provided in the **Filter** ▼ menu:

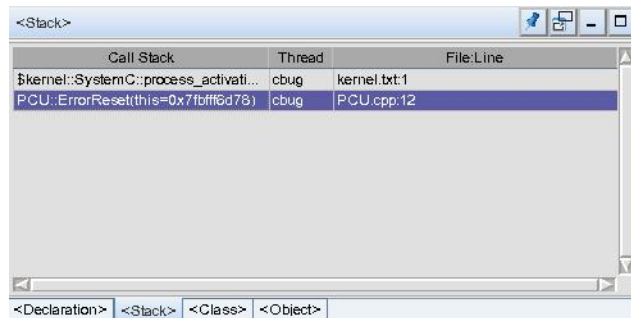
Figure 36 Filter and Search Function



Stack Pane

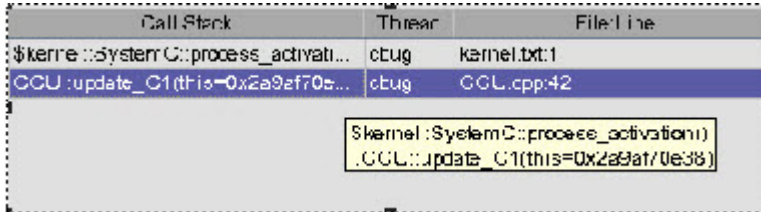
When the simulation stops inside the C/C++ code, the call stack is shown in the *Stack* pane. As shown in the following figure, the call stack information includes the call stack, thread, file name, and the corresponding source line number.

Figure 37 Call Stack Information

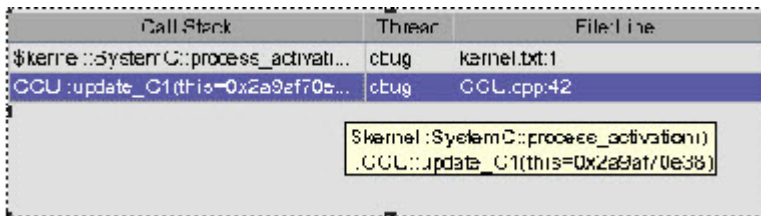


The following right-click options are provided to show more information:

- **Show Navigation Text Field:** Use the **Navigation** field to search a string in the *Stack* pane.



- **Show Tip:** Shows the stack information in a tip dialog.



- **Show File Information:** Shows the file information of the stack in the *Message* pane.
- **Display Source Code in New Tab:** Shows the source code in the source code frame with a new opened tab.

You can double-click a stack pane in the *Stack* pane to change the current stack frame. The current stack pane is the reference for the variables in the **Local** tab. The source code pane also changes the scope to the current simulator scope and shows the source code if the frame has file/line information, and no action is provided if it does not.

Call stacks are usually clipped at the SystemC or VCS kernel. Only frames belonging to user C/C++ code are shown, frames that belong to the SystemC or VCS kernel are not shown. If you want to see the entire call stack, then type the **gdb bt** command in the console.

Dynamic Dumping

The Verdi platform provides a mechanism to dynamically dump the SystemC signals to the `svtb_i` FSDB file during Interactive Debug mode. You can check the waveform of the dumped signals in the *nWave* window during the interactive simulation.

Dump and Get Signals

After the *Interactive Debug* mode is enabled, the `svtb_i` FSDB file is automatically created by Verdi. Only the `svtb_i.fsdb` FSDB file is active during *Interactive Debug* mode.

Perform the following steps to dump signals to the FSDB file and view the waveform in the *nWave* window.

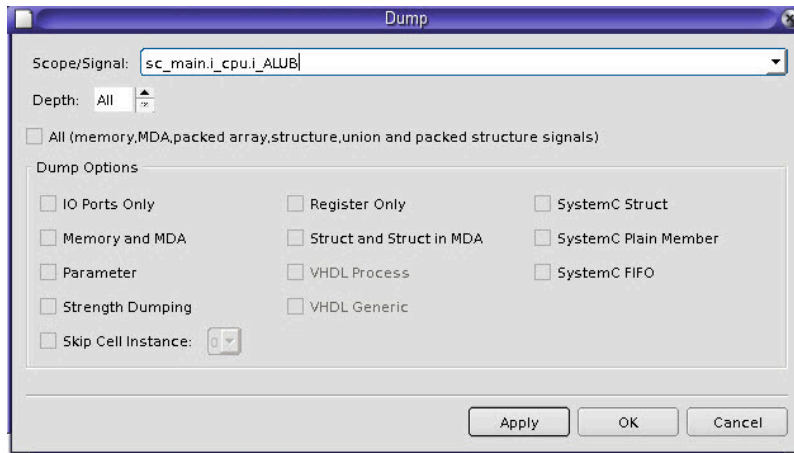
1. Select a scope in the hierarchy browser frame or select a signal in the *Signal List* pane. Use the **Dump to FSDB File** right-click option.
2. Click the  button to open the `svtb_i` FSDB file in the *nWave* window. Use the **Signal > Get Signals** command to show the dumped signals.
3. Alternatively, directly drag and drop the selected scope from the hierarchy browser frame or the selected signals from the *Signal List* pane to the *nWave* window with the loaded `svtb_i` FSDB file. The Verdi platform triggers the dynamic dumping mechanism to dump the signals if the signals cannot be found in the current FSDB file.
4. Advance the simulation to see the following values for the signals.

If a struct member is selected for dumping, the full struct is dumped to the FSDB file.

Dumping Options

You can use the **Dump** command to set the dumping configurations in the *Dump* form. Right-click an instance in the Instance pane or a signal in the *Signal List* pane and choose the **Dump** command from the menu. The selected scope or signal is automatically shown in the **Scope/Signal** field or you can directly enter the specified scope or signal (refer to [Dump Options](#)). Specify the dumping depth in the **Depth** field. For example, the specified signal is `memory_example.U0.mem[0:15]` (refer to [Dump Options](#)). Set '1' in the **Depth** field to dump the entire `memory_example` scope. Set '2' in the **Depth** to dump the `memory_example.U0`, and so on. You can set "All" in the **Depth** field to only dump the specified signal or instance to the FSDB file.

Figure 38 Dump Options

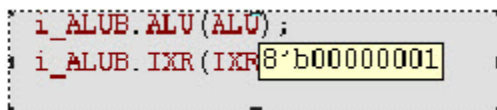


The data types of memory, MDA, packed array, structure, union, and packed structure signals can be disabled from dumping by turning the **All** option *off* to get better performance and prevent a larger waveform size. You also can enable the signal categories in the **Dump Options** field to dump the specified signal types in the scope.

Variable Value Tip

When the simulation is stopped and the option setting is done, the value of the SystemC variable can be shown when the cursor is placed on the C/C++ identifier in the source code pane, as illustrated in the following figure:

Figure 39 Variable Value Tip



Note:

- The value of the variable is based on the scope where the simulator stops. For example, there is an `i` variable both in the `func1` and `func2`:

```
void sc_class:func1(int i){
    ...
}
void sc_class:func2(int i){
    ...
}
```

Assume that the simulation is stopped in the `func2` and the value of the `i` variable in `func2` is 2. The value of 2 is displayed by the tip when the cursor is placed on any `i` variable. Variable `i` shows 2 even for `func1`.

- The tip is shown in the source code pane for all C/C++ identifiers even though the identifier is not a variable.

Setting Value-Change Breakpoint on C Variables

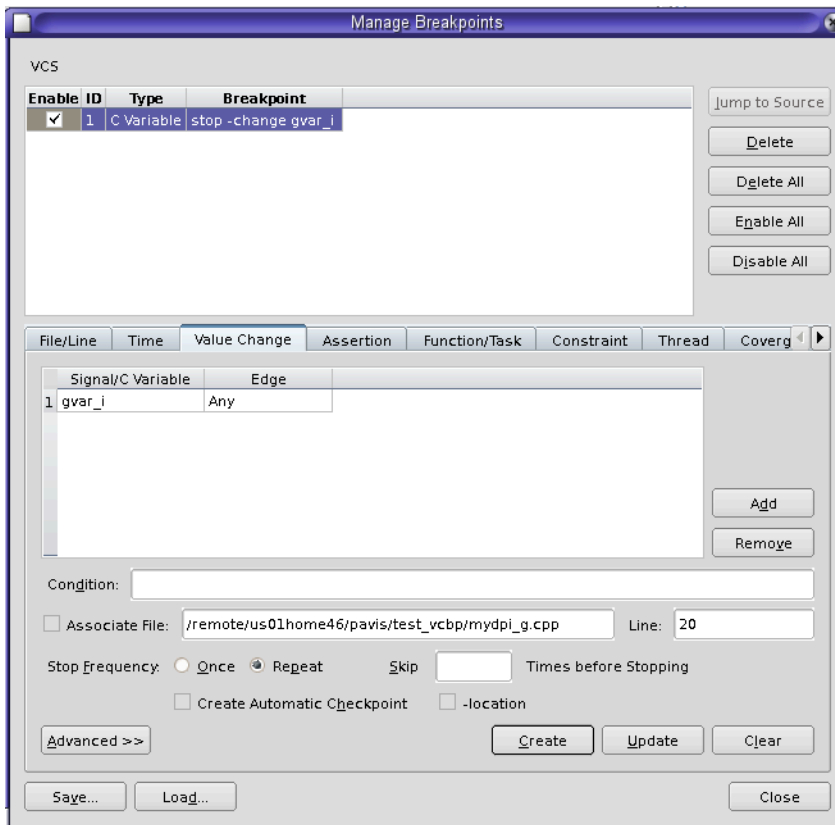
A Value-Change Breakpoint (VCBP) stops the simulation when the value of the specified variable changes. The VCBPs for C variables rely on the GDB **Watch** command and inherits the limitations. You can set C-VCBP either from UCLI or from the Verdi (and DVE) GUI.

- [Verdi GUI Usage](#)
- [UCLI Mode Usage](#)

Verdi GUI Usage

A C-VCBP can be set using the *Manage Breakpoints* form in the Verdi GUI as illustrated in the following figure. You can set the SystemC VCBP in the same manner as setting a VCBP for an HDL signal. If the current domain is a C domain (simv stopped in C, C++ or SystemC user code), the **Value Change** tab on the *Manage Breakpoints* form displays a *Signal/C Variable* column.

Figure 40 Setting a Value-Change Breakpoint for C/C++ variable



The *Manage Breakpoint* form has the following new fields and options:

New Fields and Options	Description
Associate File/Line	Specify the File or Line number. You can toggle or un-toggle the option. When you create a C-VCBP, if the option is toggled on and if the file or line is not specified, Verdi displays a warning message <i>“Associated file and line should be specified”</i> . If both are specified, Verdi calls the VCS simulator to create a C-VCBP with associated File/Line.

New Fields and Options	Description
-Location	Toggle or untoggle the option to stop at the memory location of the C Variable/Expression. When the <i>-Location</i> option is enabled, the <i>Associate File/Line</i> field is disabled. And when the <i>Associate File/Line</i> field is enabled, the <i>-Location</i> gets disabled.

The status of the existing breakpoints have the following columns:

Field Names	Description
Enable	Indicates that the displayed breakpoint of the related row is enabled.
ID	Displays the ID number of the breakpoint.
Type	Displays C Variables for C-VCBP. Displays File or Line for the associated file or line and the associated line is displayed in the Breakpoint column.
Breakpoint	Displays the details of the Breakpoint command.

Setting a Value-Change Break Point on a C-Variable

If the simulator domain is C-domain and current source window displays C/C++ file with a selected string, the *Manage Breakpoint* form is displayed if not the option remains hidden. You can invoke the *Set Breakpoint* command in one of the three ways:

- In the *Source* window, click the right-click menu and select the **Set Breakpoint** command.
- In the *Source* window line number field, click the right-click menu and select the **Set Breakpoint** command.
- In the *Watch/Local* pane, click the right-click menu and select the **Set Breakpoint** command.

UCLI Mode Usage

The section contains the following details about the C Value-Change Breakpoints (C-VCBP) feature:

- Scope of the Expressions
- C-VCBP States
- Reactivating a C-VCBP with a Different Breakpoint
- Associated Breakpoints
- Creating a C-VCBP and Line Breakpoint Pair
- -location option
- -C_expression option
- VCBP of Memory Locations
- False Positives of C-VCBPs
- C-VCBP Restart Behavior
- Supported Data Types and Hardware Limitations

A C-VCBP is specified with the existing UCLI **stop –change** command:

```
stop -change [-location] <expr>
```

The UCLI command is extended to accept local C variables or expressions - <expr>.

<expr> refers to a variable that is visible within the current C stack frame and is interpreted by SystemC debug or GDB. The simulation stops whenever the <expr> changes.

For example, in the *my_dpi.c* file:

```
001  int AA;
002  void my_DPI_func(..)
003  {
004      AA = 10;
005      AA = 11;
006  }
```

Variable AA is a global variable. It is always visible to SystemC Debug or GDB, even if the simulation has currently stopped in a different C function or even in Verilog code. Therefore, setting a VCBP is always possible.

Execution stops on reaching the next line

The execution stops when the C-VCBP reaches the next line after modifying the variables once.

For example, in the *my_dpi.c* file:

```
001  int AA;
002  void my_DPI_func(..)
003  {
004      for (int n=0; n<3; n++) {
005          AA = n;
006          AA = n+10;
007      }
008      printf("AA=...");
008  }
```

The assignment in line 5 (*AA=n*) changes the variable *AA* and stops the simv on line 6 (and not line 5). The next reachable line is frequently the next line such as line 6 in the example.

It can also be a different line. For example, the assignment in line 6 (*AA=n+10*) also stops the simv but either on line 5 if the FOR loop iterates again or on line 8 when all the FOR loop iterations are complete.

Expressions and Runtime Overhead

The *<expr>* specified as part of the **stop -change <expr>** command can either be a single variable or a complex arithmetic expression involving several variables.

If no hardware limitations occurs, the GDB watches the underlying base variables, stops when the expression changes, and evaluates it and reports a breakpoint only when the resulting expression changes. The runtime overhead depends on the frequency of the change in the base variable.

Scope of the Expressions

The C variables for which VCBP is set can either be global variables or local variables. When VCBP is evaluated for local variables, the scope of the variable ends when the scope of its function ends.

C-VCBP States

State	Description
Active	Simulation stops when the expression changes.

State	Description
Disabled	The Breakpoint command is disabled by executing the stop -disable <ID> command. The simulation does not stop when the expression changes.
Out-of-scope	The VCBP went out of scope and is not active.

The UCLI command `watch::isOutOfScope(id)` returns 1, if the C-VCBP breakpoint ID exists and is currently out-of-scope else returns 0 for all other outcomes.

UCLI command **stop -enable <id>** reactivates a C-VCBP that is out-of-scope or disabled. The command evaluates the `<expr>` in the current scope. On success it becomes active again. On failure, the C-VCBP remains or goes out-of-scope.

Simulation stops for an active C-VCBP in the following scenarios:

- When the `<expr>` changes the simulation does not stop if the breakpoint is configured with the **stop -continue <ID>** command.
- When C-CVBP goes out-of-scope and there is no associated line breakpoint that automatically activates it at later point in time.

Reactivating a C-VCBP with a Different Breakpoint

Execute the **stop -enable** command to reactive a VCBP breakpoint after it goes out-of-scope.

This feature enables combining a regular file/line breakpoint with a C-VCBP. The command is set on a regular breakpoint which reactivates the C-VCBP. The combination enables creating C-VCBP on local variables. Even if the scope of the C-VCBP ends, the next breakpoint reactivates it.

For example, in the *file my_DPI.c*

```
001 void F1() {
002     int LL; -> simulation is stopped here. We want to watch LL
...
010     if (...) LL = 17;
011     ...
...
020 }-> C-VCBP will go out-of-scope after this line
...
050 F1();
051 F1();

ucli% set line_bp [stop -line 2 -file my_DPI.c]
1
```

```
ucli% run
Stop Point #1 @10ns; -> stopped at beginning of my_DPI.c

ucli% set c_vcbp [stop -change LL] -> stop when LL changes. C-VCBP
2
```

Automatically re-activates the C-VCBP when the `F1()` is called again:

```
ucli% stop -command {catch {stop -enable $c_vcbp}} $line_bp
ucli% stop -continue $line_bp
ucli% run
```

The C-VCBP (Line number 2) of local variable `LL` is activated only as long as `F1()` runs. It goes out-of-scope as soon as `F1()` ends. But the next call of `F1()` triggers the line breakpoint (no. 1), which in turn reactivates the C-VCBP. The line breakpoint is configured such that it does not stop the simulation when it is hit. This setting makes the C-VCBP is activate during all the calls of `F1()` but goes out-of-scope between calls of `F1()`. The simulation stops only when `LL` changes. Therefore, the C-VCBP is seemingly permanent.

Management of the C-VCBP is convenient but causes some runtime overhead with each execution of function `F1()`. Stopping the simulation internally and trying to re-activate the C-VCBP requires some CPU time and slows down the simulation.

Also note that the `<expr>` of the C-VCBP is evaluated each time the line breakpoint is hit and re-activates the C-VCBP. This is beneficial in case the memory location of `<expr>` keeps shifting, if for example, it is a local variable on the stack and the function is called through different call stacks. But if you need to watch changes of a specific object (at a stable memory location), then use the **-location** option .

Associated Breakpoints

A new option for the UCLI **stop** command associates an arbitrary breakpoint with a C-VCBP.

```
ucli% stop -associate <bp> <vc>
```

Each breakpoint has atleast one associated breakpoint.

Execute the following command to un-associate:

```
ucli% stop -associate {} <vc>
```

A line breakpoint may have many C-VCBP associated with it. You can set a breakpoint on the first line of a local variable in a function for the purpose of reactivating all the C-VCBPs.

Associating a breakpoint has the following effects:

- The C-VCBP is configured to make the simulation continue even after going out-of-scope.
- The breakpoint is configured to reactivate the C-VCBP and then continue.

Enabling or disabling or deleting a VCBP changes the associated breakpoints.

Creating a C-VCBP and Line Breakpoint Pair

The following UCLI routine combines creation of both the line breakpoint and C-VCBP and creates an association between them:

```
watch::create_persistent_c_vcbp -change <E> -file <F> -line <L>
```

For example,

```
ucli% watch::create_persistent_c_vcbp -change LL file my_file.c -line 100
```

The example creates a line breakpoint at <F><L> on line 100 and creates a C-VCBP for <expr> for the expression LL and associates both the breakpoints. The return value is a list of both breakpoint IDs {<line-id><c-vcbp-id>}

If setting of either of the breakpoint fails, then no breakpoint is created and an error is returned.

If there is a line breakpoint existing in the same line or file location and if it is associated with another C-VCBP, then the existing line breakpoint is reused for the new C-VCBP. Therefore, creating a set of C-VCBPs with different expressions on the same file or line location results in a single breakpoint and a set of C-VCBPs associated with the same line breakpoint.

Therefore, it is recommended that this routine is used only with local variables that go out of scope and recommended not to use this with global variables or memory locations.

Note:

The **-location** option is not supported in the
`watch::create_persistent_c_vcbp` routine.

Automatic reactivation of C-VCBP slows down the simulation if it occurs frequently. The SystemC Debug displays the following message in such a case:

```
Note-[CBUG-BP-17] Persistent C-VCBPs are active
One or more persistent C value-change breakpoints are frequently
triggering a cycle of going out of scope and re-activation by a
controlling line breakpoint. Such cycles may cause a noticeable slow
down the simulation. This message is printed only once every 1000
cycles.
If these cycles are not intentional, then type 'stop' to identify
and disable the relevant breakpoints. Look for 'controlling VCBPs'.
```

Change the limit to report this message by changing TCL variable 'watch::vcbp_silent_oos_limit'.

You can set the limit of automatic reactivations using the following routine:

```
watch::vcbp_silent_oos_limit
```

The default limit is *1000* cycles.

You change the limit as follows:

```
ucli% set watch::vcbp_silent_oos_limit 20000
```

-location option

The issue with C-VCBP is its tendency to get disabled when the expression goes out-of-scope. You can either re-activate it or use the **-location** option. The **-location** option is available only for C-VCBP and not for Verilog or HDL or SystemC signals. The **-location** option has the following syntax:

```
ucli% stop-change <expr> -location
```

The underlying GDB command monitors a memory location and stops when the value in that memory changes. The <expr> is evaluated in its resulting memory address. With the **-location** option, <expr> is evaluated exactly once when the C-VCBP is set for the first time. The resulting memory address is watched, <expr> is ignored thereafter.

The benefits and penalties are illustrated in the following example:

Example 1 *my_file.c*

```
101  struct Object {
102      int ID;
103      double A;
104      bool B;
105      Object(int id_) : ID(id_),A(0.0),B(false) {}
106  };
...
201  Object* lookup_entry(int key)
202  {
203      Object* P = <search list for entry matching key>
204      if (P)
205          return P;                ç simulation is stopped here
206      else
207          ...
208  }
```

Set the C-VCBP:

```
ucli% get key
17
ucli% get P -radix hex
0xffff0010
```

```
ucli% get &(P->B) -radix hex
0xffff0018
ucli% stop -change P->B -location
1
ucli%
```

The VCBP remains active until the simulation ends. The simulation stops when the new value is written to memory address 0xffff0018. The expression `P->B` becomes meaningless but the memory address remains under watch.

The advantages of the **-location** option is as follows:

- The C-VCBP is hit only when the object at the memory address 0xffff0018 is modified, it does not stop for any other object, (assuming that the object with the `key=17` continues to exist at address 0xffff0018 through the rest of the simulation).
- The C-VCBP remains active even after the `lookup_entry()` function ends and `P->B` goes out of scope.

-C_expression option

You can set a C-VCBP as follows:

```
ucli% stop -change <expr>
```

The `<expr>` refers to a Verilog/VHDL/SystemC signal and also to a plain C expression. The ambiguity regarding the language of the expression is resolved as follows:

- If `-c_expression` option is present, then `<expr>` must refer to a C expression.
- If the `<expr>` matches a Verilog, VHDL or SystemC signal, then the VCBP is set on the signal. A matching C expression, if any, is ignored.

If no `-c_expression` is present, `<expr>` does not match a Verilog, VHDL, or SystemC signal but matches a C expression, then a C-VCBP is created.

VCBP of Memory Locations

The `<expr>` of a C-VCBP also refers to a memory address in which case a word (4 bytes) starting at `<expr>` is watched for changes.

For example,

```
CBug% stop -change *0xffff0018
```

Setting a VCBP on an address is similar to using the **-location** `<expr>`. With the **-location** option, SystemC Debug is aware of the `<expr>` type and the number of bytes of memory allocated to it, hence the underlying GDB watchpoint is set precisely.

The benefit of using a memory is its robustness and availability wherever the simulation has stopped.

C-VCBPs on memory locations can always be activated again after a restart. But this is also a penalty because the memory may frequently be used for different purposes during simulation startup and thus create frequent false positives. Both the approaches can be combined with the following command:

```
CBug% stop -change *(my_object_type*)0xffff0018
```

with `my_object_type` referring to the type of expression `lookup_entry(17)->A`.

False Positives of C-VCBPs

When an object at a memory location is deleted, freed and reallocated to a another object, VCBP remains unaware of this change and continues to function in the same manner. This results in a false positive and it is likely to occur when a C-VCBP watches a memory location.

In the example, object P is freed and its memory re-allocated to string STR:

```
...
301  Object* P = new Object(17); // P1=0xffff0010
...      ? stop -change -location P->A
311  P->A=true; ... // BP hits (good)
...
330  delete P; // mem 0xffff0010 if freed
331  char* STR = new char[100]; // mem 0xffff0010 allocated again
332  strcpy(STR, ...); // BP hits (bogus)
```

If the object at 0xffff0010 is deleted, its memory is freed and reallocated again to some unrelated function, the C-VCBP remains unaware of these changes. It keeps watching the memory location 0xffff0018. Any modification, results in a C-VCBP to hit.

The C-VCBP hits in line 311 (simv stops in the next line, 312) as required when the memory still belongs to the object with ID=17. But, the C-VCBP also gives a bogus hit at line 332 (simv stops in 333) which is confusing. The C-VCBP is technically correct to stop because memory location 0xffff0010 is modified and **-location** option specifies to watch the memory location rather than the original expression `P->A`.

False positives are especially likely when a C-VCBP watches a memory location.

C-VCBP Restart Behavior

The C-VCBP behaves in the following manner, after a simulation restart:

- C-VCBP with an associated file/line breakpoint is always set to out-of-scope status. Such a C-VCBP is used for a local variable which definitely goes out-of-scope, after a restart. The C-VCBP is automatically enabled after the associated file/line breakpoint is hit at a later point.
- C-VCBP with **-location** option remains disabled upon restart. The **-location** option is used for context sensitive expressions. Since the context is not available directly after a

restart, you can manually enable the C-VCBP again when the simulation is back in the given context.

- The system tries to enable any other type of C-VCBP. When the enabling fails, the C-VCBP goes out-of-scope.

Supported Data Types and Hardware Limitations

The C-VCBP works on any type of C/C++ variable but there are hardware limits on the number of bytes that the GDB can watch. Therefore, most native SystemC types cannot be supported.

Note:

Native SystemC types. For example, `sc_in`, `sc_signal`, `sc_int` are not supported by C-VCBPs.

Hardware Limitations

Hardware watchpoints are implemented based on hardware feature which has machine dependent limitations. Therefore, the limit for hardware watchpoint varies with the host/CPU being used.

If the hardware limit has reached when a C-VCBP is set, then the UCLI command fails with the following error message:

```
CBUG-VCBP-C-1 cannot set C value-change breakpoint
Value-change breakpoint of expression '<expr>' cannot be set as a GDB
hardware watchpoint. The expression may be too complex or require too
many bytes in memory. If this is a single variable, then use command
'get -cbug {sizeof(<expr>)}' to see how many bytes are needed. Use a
different expression that uses fewer bytes, for example only a single
array element or only a single member of a struct.
```

Even though the individual watch points are small, all the watchpoints combined exceed the limit when the simulation is about to continue because of a **Run** or **Step** command.

The following error message is displayed on encountering such a scenario:

```
Error-[CBUG-BP-11] Too many C value-change breakpoints
Too many value-change breakpoints of local C expressions are active
and breakpoint(s) '1 3 ' could not be activated. The simulation cannot
continue, commands such as 'step' or 'run' are not possible.
Disable/delete these or other value-change breakpoints of local C
expressions.
```

SystemC Data Types and Limitations

C-VCBP works with any type of C/C++ types as long as the variables are within the GDB/CPU limitations. But in most cases, when a SystemC data type exceeds the CPU/GDB limitation, an error message is displayed.

For example,

```
ucli% gdb whatis s_int
sc_core::sc_signal<int>
```

```
ucli% print s_int
17
```

```
ucli% stop -change s_int
```

The SystemC debug displays the following error message:

```
Warning-[CBUG-BP-16] Limited support for native SystemC types
Expression 's_int' is of native SystemC type 'sc_core::sc_signal<int>'. C
value-change breakpoints give only limited support in this case:
- SystemC ports and signals are frequently too large (CBUG-BP-11)
- Values will be printed in GDB syntax and not with the usual
  pretty-print
  syntax
- Several types will only show '<incomplete type>' as new value
- The simulation will also stop at destructors of base classes
If this is an SystemC port, signal or other channel, then specify the
object with its full pathname. It will implement the value-change
breakpoint by means of a callback from the SystemC kernel instead of a
GDB 'watch' command.
```

```
Error-[CBUG-BP-13] Software value-change breakpoint forbidden
Expression 's_int' is valid, however, GDB can create only a software and
not a hardware watchpoint for it. No breakpoint has been created because
software watchpoints would drastically slow-down the simulation. Use
command 'cbug::config allow_sw_vcbp on' to enable software value-change
breakpoints and try again.
```

Using HDL Value-Change Breakpoints to Stop a SystemC signal, Port, or Channel

To stop the simulation when a SystemC signal, port or channel changes, use an HDL value-change breakpoint instead of C value-change breakpoint.

For example:

```
ucli% stop -change top.inst0.s_int
```

Do not use **-c_expression**, use the full pathname.

5

Known Problems and Limitations

The following are the known issues and limitations in SystemC Debug:

- Bit Field is not supported during the dumping operation.
- Verdi SystemC Debug does not support SystemC RIDB/signal dumping for VCS slave simulations (`vcs -sysc ... -slave ...`).
- The action of adding or dragging and dropping of a SystemC signal from the source code pane into the *nWave* pane is not supported.
- Array in struct data type is not supported with FSDB dumping.
- If an FSDB file contains SystemC signal(s), it is not supported by FSDB utilities and virtual FSDB files.
- If the design is a SystemC-on-top design, Verdi shows a scope `sc_main` in the hierarchy tree, even if `sc_main` is a C function and not a SystemC instance. The following are the limitations of the `sc_main` scope only for SystemC-on-top designs:
 - The signal list shows only `sc_signals` but no plain members.
 - Source names are not visible for SystemC objects in the `sc_main` scope. Only OSCI names are visible.
 - Double-clicking a SystemC object in the `sc_main` scope does not jump to the corresponding source code.
- The Show Calling in *nTrace*: the **Show Calling** command shows the place in the parent where the selected module instance is instantiated. The **Show Calling** for SystemC instances has several limitations:
 - Supported: SystemC instance inside a SystemC module, no pointers, OSCI, and source name match.

For example:

```
SC_MODULE(sctop) {
    sc_driver sub_inst;
    SC_CTOR(sctop) : sub_inst("sub_inst") ...
};
```

- Not supported: SystemC-inside-SystemC but with usage of pointers.

For example:

```
SC_MODULE(sctop) {
    sc_driver* sub_inst;
    SC_CTOR(sctop) {
        sub_inst = new sc_driver("sub_inst");
    }
};
```

- Not supported: SystemC-inside-SystemC, OSCI, and source name do not match.

For example:

```
SC_MODULE(sctop) {
    sc_driver sub_inst;
    SC_CTOR(sctop) : sub_inst("sub") ...
};
```

- Not supported: module instances under *sc_main* function.
- Source names: Verdi/CBug can recognize source names only for members of classes derived from *sc_module*. Local variables are not supported.

For example:

```
SC_MODULE(sctop) {
    sc_signal<int> S1; ȳ supported: port_0(S1)
    sc_signal<int> S2*; ȳ supported: port_1(S2)
    SC_CTOR(sctop) {
        S2 = new sc_signal<int>();
        sc_signal<int>* S3 = ȳ not supported: port_3
            new sc_signal<int>(); local var, not a class member
        new sc_signal<int>(); ȳ not supported: port_4

        not variable at all
    }
};
```

- For the trace driver/load/connectivity features for HDL signal in mixed-language design, tracing stops when a SystemC scope is reached and the name of this scope is meaningless.
- For fixed point limitations:
 - Cannot change radix in current stage.
 - Cannot do few GUI operations such as **Signal > Logic Expression** and **Waveform > Analog Waveform**, **Invert Waveform**, and so on.

- For the *nSchema* window:
 - No connection for the SystemC relative objects
 - SystemC processes are not supported
 - The tracing function does not work for SystemC objects
 - Active Annotation works for SystemC signals only in post-process debug but not in interactive mode
- For the *nWave* window, typing the correct OSCI name of a signal in the **Find Signal** field of the *Get Signals* form and pressing the return key can add the signal to the *nWave* window directly. Typing the source name causes the search result to be shown but it is not added to the *nWave* window directly.
- For the *nCompare* module, SystemC signals are not supported.

The error message is as follows:

```
*WARN* fsdb2vcd doesn't support system c FSDB
```

- The following simulation commands are not supported by the simulator when the simulator is in the C domain or the C/C++ debugging function is enabled. These commands are not disabled or hidden. When you try to use these commands (by clicking the icons or selecting from the menu), Verdi sends the command to simulator and displays a warning message in the *Interactive Console* window if there is a simulator warning:
 - Step in Thread
 - Step in Testbench
 - Step in Constraint Solver
 - Redo Randomize Call and Step in Constraint Solver
 - Add Checkpoint
 - Rewind to Checkpoint
 - Delete Checkpoint
 - Save State
 - Restore State

- Following are the limitations when KDB elaboration is performed along with the VCS elaboration in a sequence to avoid the delay when Verdi is invoked for the first time:
 - If SystemC constructors are effected by simv runtime arguments, then you must specify the arguments with the `-syscelab` option as follows:

```
% vcs ... -syscelab arg1 ... -syscelab arg2
```
 - If SystemC constructors are effected by environment variables, then set the environment variables.
 - If SystemC constructors are effected by files in the current working directory, then the files must be available during elaboration.
 - The `-kdb=sysc_elab` option which is used to avoid the delay when Verdi is invoked for the first time does not work with `-sysc=adjust_timeres`, `-syscelab`, and `-sysc=shared`, options.